TUNABLE NON-LINEAR MANIFOLD ROMS FOR PDES VIA MATRIX-FREE PETROV–GALERKIN PROJECTION

Anonymous authors
Paper under double-blind review

ABSTRACT

A trained neural operator gives one accuracy–cost trade-off per model, and changing it means retraining.[1] We present a nonlinear-manifold reduced-order model (NM-ROM) whose balance between accuracy and speed is chosen when the model is used. Its *spatial bank*, a set of learned functions of space, is ordered once by importance after training, so a query chooses how many bank functions to use and whether to solve with a small nonlinear head or in the linear span of the bank, without changing a learned weight. Every linear term is precomputed exactly, the discrete PDE residual is solved by least-squares Petrov–Galerkin projection onto fixed tests, and the nonlinear term is evaluated on a sparse set of nodes. We compare every setting with the fastest tested solver that reaches the same accuracy; on the new problems that solver may choose its mesh, integrator and step. On a three-dimensional Navier–Stokes family with sharper interacting vortices and lower viscosity, on 128 fresh test cases and three independently trained models, the head reaches 0.28–0.42 % worst-case error at bank width 192, and a faster setting 0.35–0.48 % at 5.1–10.2× the speed of the matched solver from a 450-configuration ladder; the linear span of the same bank is unstable (7.7–70 %). On unsteady flow past a cylinder, one trained bank meets six acceptance targets at a held-out Reynolds number at widths 16 to 32, with a rollout 25–54× faster than the cheapest finite-element run that meets the same targets. The advantage appears where the matched solver needs a fine mesh and the solution family is low-dimensional; on constant-coefficient box problems transform solvers are mostly faster, and we use those problems as correctness checks.

1 INTRODUCTION

Two lines of work aim to make partial differential equation (PDE) solutions fast to compute: neural operators and reduced-order models (ROMs), and both have advanced quickly. Neural operators (Li et al., 2021; Lu et al., 2021; Kovachki et al., 2023) now include attention-based models (Wu et al., 2024; Luo et al., 2025) and pretrained foundation models (Herde et al., 2024; Hao et al., 2024; Holzschuh et al., 2025), and are fast to evaluate once trained (Azizzadenesheli et al., 2024). In parallel, nonlinear-manifold ROMs (NM-ROMs) have moved from dense autoencoders (Lee and Carlberg, 2020; Y. Kim et al., 2022) to domain-decomposed (Diaz et al., 2024), neural-field (Chen et al., 2023; M. Kim et al., 2024; Berman and Peherstorfer, 2024) and hyper-reduced Navier–Stokes models (Romor et al., 2025), while keeping the projection of the governing equations used by linear projection-based ROMs (Benner et al., 2015; Carlberg et al., 2011). Hesthaven et al. (2026) survey these nonlinear methods for transport-dominated problems.

Two properties decide whether such a model can stand in for a full-order solver: whether accuracy can be traded for speed when the model is used, and whether it is faster than a well-chosen solver of the same accuracy. Full-order methods (FOMs) have the first property, since their mesh, tolerance and time step are set per query (each new problem instance to solve). A trained neural operator gives one trade-off between accuracy and wall-clock time at a given mesh, and changing it in general requires retraining (de Hoop et al., 2022); comparisons against weak numerical baselines can also overstate their speed (McGreivy and Hakim, 2024). Linear ROMs can require many basis functions for moving

[1] Revised version prepared during the author–reviewer discussion. Every changed or added passage, table and figure is set in blue; passages moved to the appendix or deleted are not marked.

fronts (Cohen and DeVore, 2015; Peherstorfer, 2022). NM-ROMs are more compact, but their online nonlinear solve can offset the benefit of fewer unknowns. We ask: *can one trained model offer an accuracy choice at query time, and on which problems is the resulting reduced model faster than a solver of the same accuracy?*

We answer this question on a three-dimensional Navier–Stokes family that a coarse mesh no longer resolves, on unsteady flow past a cylinder, on viscous Burgers in two and three dimensions, and, as correctness checks, on Poisson and heat problems. Our NM-ROM builds the solution from a *spatial bank*, a set of R functions of space, each a small neural network of the coordinates, whose weighted sum gives the solution; the weights come from a small nonlinear *head*. After training, the bank is ordered once by how much of the training data each of its functions carries. A query then chooses how many of the ordered functions to use, R', and whether to solve with the nonlinear head or in the linear span of those functions, without changing a learned weight. Because each bank function is a small network of the coordinates, for Poisson, heat and Burgers the learned functions are shared by every mesh (the Navier–Stokes results of Table 1 use per-mesh POD banks); the reduced operators are assembled once per mesh, and we charge that assembly as an offline cost. On periodic boxes a POD basis interpolated across meshes transfers as well as the coordinate network, so there the network provides a single mesh-free object rather than higher accuracy. Our intended use is repeated queries within a parameterised family after offline learning and assembly. The learned network provides a compact representation; each query still determines its coefficients from the PDE residual, without access to the reference solution.

The main results are Tables 1 (harder Navier–Stokes), 2 (cylinder) and 4 (neural-field ROMs and neural operators). Classical-solver costs are accuracy-matched; neural-operator comparisons report prediction errors under the stated training budgets.

Contributions.

1. **Accuracy chosen at query time from one trained NM-ROM:** After training, we order the spatial bank once by importance. A query then chooses how many of the ordered bank functions to use (the width R') and whether to solve in their linear span or with the nonlinear head, without changing a learned weight, so one trained model gives a ladder of accuracy–cost settings (Figure G.1); every row of Table 2 comes from one trained bank.
2. **A nonlinear head that makes a small model accurate:** The bank/head factorisation makes every linear term an exact precomputed matrix, and the head constrains the dynamics to a low-dimensional manifold inside the bank's span. On the harder Navier–Stokes family the head reaches 0.28–0.64 % at bank widths 128–192 where the linear span of the same bank gives 7.7–70 %; on Burgers, at 16 coordinates, the head gives 2.56 % against 56.9 % for a linear and 31.8 % for a quadratic closure on the same bank.
3. **Speed measured against accuracy-matched solvers, with its limits:** The model is faster than a matched solver where that solver needs a fine mesh and the solution family is low-dimensional (harder Navier–Stokes, 3.3–11.7× across settings; cylinder, 25–54×). On the original Navier–Stokes family a solver free to coarsen its mesh gives 0.63/1.65/0.94×, on box Poisson and heat 3D transform solvers are faster than our model, and on Burgers 2D the fast setting is slower than its matched solver at 256^2 and 512^2; we report all of these (§5.3). PCA ordering, operator precomputation and quadrature are established; what is new is their tested combination.

2 Related Work

Projection-based ROMs. Linear ROMs approximate solutions in a fixed subspace (Benner et al., 2015; Sirovich, 1987); moving fronts can require a large basis (Cohen and DeVore, 2015; Greif and Urban, 2019; Peherstorfer, 2022). Nonlinear-manifold ROMs replace the subspace by a learned manifold and keep the residual projection (Lee and Carlberg, 2020; Y. Kim et al., 2022; Romor et al., 2023; Diaz et al., 2024), including quadratic manifolds (Geelen et al., 2022; Barnett and Farhat, 2022), neural-network corrections to a linear basis (Barnett et al., 2023), adaptive bases (Peherstorfer and Willcox, 2015; Carlberg, 2015), hyper-reduced models of Navier–Stokes flows (Romor et al., 2025) and graph autoencoders on unstructured meshes (Magargal et al., 2025). These methods solve the PDE online; a linear or quadratic basis is stored on the mesh it was built on, though it can be interpolated to another, and an autoencoder decoder is evaluated at every node in each online

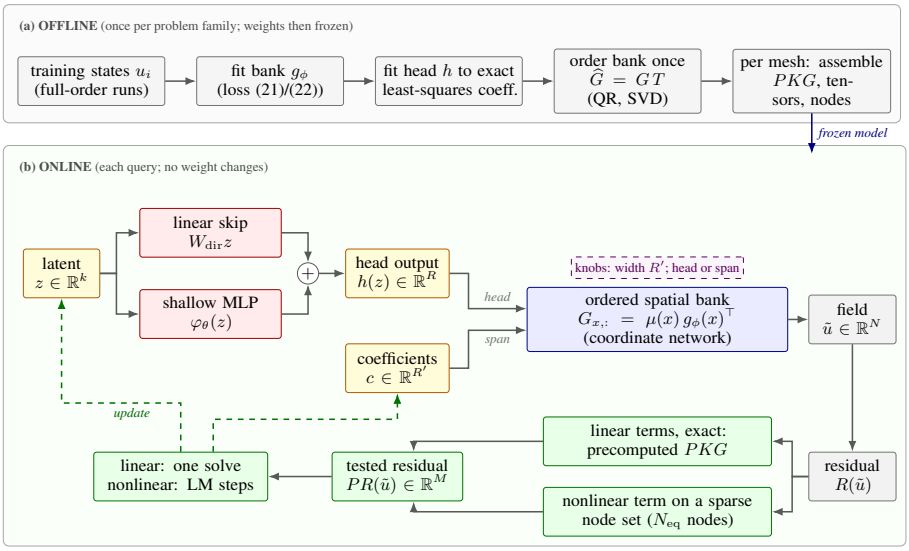


Figure 1: NM-ROM, redrawn with the two stages separated. **(a) Offline, once per problem family:** fit the coordinate-network bank to full-order training states (Appendix C.1), fit the head to their exact least-squares bank coefficients, order the bank once, and assemble tests, reduced operators and nonlinear-term nodes for each mesh; all of it is charged as offline cost (Table C.4). **(b) Online, each query:** the head maps a latent code z to bank coefficients, $\tilde{u} = G\,h(z)$, or the span of the first R' ordered columns gives $\tilde{u} = \widehat{G}_{R'}c$; the residual, tested against M fixed tests P with precomputed linear terms and a sparse node set for the nonlinear term, is solved by one least-squares solve (linear PDE in the span) or Levenberg–Marquardt steps. Dashed tag: choices made at query time.

iteration unless it is hyper-reduced. Latent-dynamics ROMs instead learn the reduced dynamics from data (Fries et al., 2022; He et al., 2023). Least-squares Petrov–Galerkin projection (Carlberg et al., 2011) and preassembly of reduced operators (Ştefănescu and Sandu, 2014; Weder et al., 2024) are established components that we reuse. Our closest relatives are neural-field ROMs (Chen et al., 2023; M. Kim et al., 2024; Berman and Peherstorfer, 2024), which also evaluate a coordinate network at individual nodes; we compare with a CROM-style and a CoLoRA-style model in §5.4. We differ in three ways: our decoder separates space from the latent code (a bank times a head), so every linear term is precomputed exactly; the bank carries an exact Dirichlet factor (on the cylinder, a product of smooth distance factors to the boundary pieces, with the inflow carried by a lift function); and the bank is ordered after training, so its nested widths give accuracy settings at query time from one trained model, as runtime-tunable networks do for vision and language models (Yu et al., 2019; Cai et al., 2020; Kusupati et al., 2022; Devvrit et al., 2024).

Neural operators. FNO (Li et al., 2021), DeepONet (Lu et al., 2021), U-Net (Ronneberger et al., 2015; Takamoto et al., 2022) and Transolver (Wu et al., 2024; Luo et al., 2025) learn solution maps, as do neural-field operators (Serrano et al., 2023) and pretrained PDE foundation models (Herde et al., 2024; Hao et al., 2024). Their resolution or inference procedure can change accuracy and cost; our accuracy setting instead changes the trial space in which the PDE residual is solved.

Hyper-reduction and baselines. Empirical quadrature (Hernández et al., 2017; Yano and Patera, 2019; Hernández et al., 2024) and energy-conserving sampling and weighting (ECSW) (Farhat et al., 2014) reduce nonlinear residual evaluation through non-negative weights; we describe that construction, test every rule, fitted or not, on the states the solver reaches, and use a fixed uniform sub-lattice where it suffices (§3.3). Because learned solvers are often compared with weak baselines (McGreivy and Hakim, 2024), we compare each setting with the fastest configuration of a solver ladder that is at least as accurate, including coarser meshes and, on box problems, sine-transform solvers.

3 METHODOLOGY

A full-order solve has one unknown per grid node, so its cost grows with the mesh size N. For each query we instead solve the same discretised PDE in a few unknowns, at a cost that barely depends on N apart from forming the output field. This needs three ingredients. First, a compact set of candidate solutions, the trial manifold (§3.1): the solution is a weighted sum of the functions in the spatial bank, and its weights are either produced by the nonlinear head from a few latent unknowns or solved for directly on the first R' bank functions. Second, a way to choose the right candidate without knowing the answer (§3.2). Third, a way to evaluate the PDE without visiting all N nodes (§3.3): linear terms are computed once offline, and the nonlinear term is sampled at a few nodes (in three dimensions, it is evaluated with a precomputed quadratic tensor). Figure 1 shows the whole pipeline. Throughout, $u \in \mathbb{R}^N$ is the full-order discrete state on a uniform grid and $K \in \mathbb{R}^{N\times N}$ is the sparse finite-difference matrix of $-\Delta$.

3.1 TRIAL MANIFOLD WITH EXACT DIRICHLET ENFORCEMENT

Solutions of one PDE family share their spatial structure, so a few learned functions of space can describe all of them; the nonlinear head lets their combination follow features, such as moving fronts, that a small fixed combination misses. We do not use an encoder. The decoder writes the state as a fixed set of spatial functions times coefficients produced by a small network,

$$\tilde{u}(z) = \mathcal{D}(z) = G\,h(z), \qquad G_{x,:} = \mu(x)\,g_\phi(x)^\top, \qquad \mu(x) = 16\,x_1(1-x_1)\,x_2(1-x_2). \tag{1}$$

Each column of the bank $G \in \mathbb{R}^{N\times R}$ is one spatial function. The function comes from a small network g_ϕ that takes the coordinates of a point and returns a value there, so a column is simply that network evaluated at every grid node. The head $h : \mathbb{R}^k \to \mathbb{R}^R$ takes the latent state z and returns one weight per column. Because μ is zero on the boundary Γ, every column of G is zero there too. The boundary values therefore never change when z changes, on any mesh, and the boundary equations do not enter the solve. The cube and the L-shaped domain define their own boundary factor (Appendix A), zero on their boundaries in the same way. On the cylinder the factor is a product of smooth distance factors to the inlet, the channel walls and the cylinder surface, which can be formed for any boundary whose distance can be computed; the inflow is carried by a lift function and the bank represents the remainder (Appendix E.2). Each query then solves for z (head) or c (span) by solving the PDE (§3.2).

Ordering the bank. Training does not rank the columns of G, so after training we replace them by combinations sorted from most to least important,

$$\widehat{G} = G\,T. \tag{2}$$

The matrix T comes from a principal component analysis of the training fields written in the bank (Appendix A.1). Keeping the first R' columns gives the best R'-dimensional subspace of the bank for the training data. At full width nothing changes: $\widehat{G}$ spans the same space as G, and no weight is retrained. A query then solves in one of two ways,

$$\tilde{u} = G\,h(z), \quad z \in \mathbb{R}^k \quad \text{(head)}, \qquad \tilde{u} = \widehat{G}_{R'}\,c, \quad c \in \mathbb{R}^{R'} \quad \text{(span)}, \tag{3}$$

where $\widehat{G}_{R'}$ holds the first $R' \le R$ columns. The head is a nonlinear manifold with k unknowns; the span is a linear space with R' unknowns. The width R' is the accuracy knob. The columns are nested, so narrowing the span needs no retraining, and cheaper to solve and to convert back to a field.

3.2 PROJECTING THE PDE ONTO THE MANIFOLD

At query time the solution is unknown but the PDE is known, so we choose the candidate that best satisfies the discretised equations; no solution data is needed at query time. Each PDE we consider gives a discretised residual $R(u)$ that vanishes at the discrete solution. We substitute the trial manifold into R and, instead of pulling the residual back through the decoder Jacobian, test it against M fixed vectors, the rows of $P \in \mathbb{R}^{M\times N}$. On the square these are the M lowest sine modes, which are eigenvectors of K. A head query solves

$$z^\star = \arg\min_z \ \tfrac{1}{2}\,\|P\,R\big(\tilde{u}(z)\big)\|_2^2; \tag{4}$$

a span query solves the same problem in c. We choose M with the width: $M \ge k$ for the head and $M \ge R'$ for the span ($M = 4R'$ for Burgers). This is least-squares Petrov–Galerkin projection

(Carlberg et al., 2011; Lee and Carlberg, 2020) with fixed rather than state-dependent tests; each row of PR is also scaled by a fixed factor that puts the tests on a common scale (Appendix A); we leave this scaling out of the notation. We solve (4) by damped Gauss–Newton (Levenberg–Marquardt) (Marquardt, 1963),

$$\Delta z = -\big(J^\top J + \lambda\ \mathrm{diag}(J^\top J)\big)^{-1} J^\top P\, R\big(\tilde{u}(z)\big), \qquad J = P\,\frac{\partial R}{\partial u}\,J_{\mathcal{D}}(z), \tag{5}$$

and keep a step only if the residual decreases. Here $J_{\mathcal{D}}(z) = G\,J_h(z)$ for the head and $J_{\mathcal{D}} = \widehat{G}_{R'}$, a constant, for the span. Unlike $J_{\mathcal{D}}(z)^\top$, the tests P do not change with z, which is what lets every linear term be computed once (§3.3). For Poisson in the span, (4) is a linear least-squares problem solved once, and for heat in the span each step is linear; the iteration is needed only for the head and for the nonlinear PDEs.

Elliptic (Poisson). In the span the Poisson residual $R(u) = Ku - F$ is linear in c, so a query is one least-squares solve with the fixed matrix $PK\widehat{G}_{R'}$ (Appendix A.2).

Time-dependent problems. Heat (Crank–Nicolson), Burgers (backward Euler with a sign-upwind stencil) and Navier–Stokes (implicit midpoint on the periodic cube) each give a residual $R(u_{n+1})$ for one time step; their forms are in Appendix A. Each step solves

$$z_{n+1} = \arg\min_z \|P\,R\big(\tilde{u}(z)\big)\|_2, \qquad u_n = \tilde{u}(z_n), \tag{6}$$

by (5), warm-started from the previous step; the span solves the same problem in c_{n+1}. The heat residual is linear, so in the span the rollout applies one fixed $R' \times R'$ matrix per step. Burgers and Navier–Stokes add a nonlinear advection term $A(u)$, so both the head and the span iterate (5) at every step. For Navier–Stokes the tests P are divergence-free Fourier modes, so the pressure drops out of PR; because the vortices travel, the bank is evaluated in a frame centred on the flow whose shift is solved at each step (Appendix A.5).

3.3 Hyper-reduction

The manifold reduces the unknowns from N to k or R', but the tested residual $P\,R(\tilde{u})$ still visits every node, so each evaluation costs $O(NR)$, which would erase the wall-clock benefit. We remove this cost in two ways, one for linear terms and one for the nonlinear term.

Linear terms: computed once and exact. A linear term is tested once, offline. For the Laplacian,

$$PK\tilde{u} = \big(PKG\big)\,h(z), \qquad PKG \in \mathbb{R}^{M\times R}, \tag{7}$$

and the span uses $PK\widehat{G}_{R'}$, the first R' columns of $PKGT$. Online a linear term costs $O(MR)$ and is exact. For Poisson and heat this is the whole residual, so their reduced solves make no approximation beyond the trial space.

The nonlinear term: Empirical Quadrature. The advection terms of Burgers and Navier–Stokes cannot be computed once. We use Empirical Quadrature (Hernández et al., 2017; Yano and Patera, 2019) to select a non-negative per-node weight vector w, independent of the state, such that

$$PA(\tilde{u}) = \sum_{i=1}^{N} P_{:,i}\,A_i(\tilde{u}) \;\approx\; \sum_{i\in\mathcal{S}} w_i\,P_{:,i}\,A_i(\tilde{u}). \tag{8}$$

It is fitted by non-negative least squares on states the solver reaches on training trajectories (Appendix A.6), giving a support $\mathcal{S}$ of $N_{\mathrm{eq}} \ll N$ nodes. Where a simpler rule passes the same check we use it instead: the Burgers 2D rows of Table 3 use a fixed uniform 63×63 interior lattice with equal weights and no fit, which reduces the query at 1024^2 from 1,263 to 49.3 ms at unchanged error. The decoder is evaluated only on $\mathcal{S}$ and its stencil neighbours, $\mathcal{S}^+$; those rows of the bank are computed once and cached.

Cost. With linear terms computed once and advection on $\mathcal{S}^+$, one iteration of (5) costs

$$\mathcal{O}\big(MR\,(N_{\mathrm{eq}} + k)\big), \tag{9}$$

with k replaced by R' in the span, plus the cost of the head and of the small normal-equation solve, none of which depends on N. Returning the field at an output time is one further $O(NR')$ product.

Checking the rule. Each rule is checked on the states the reduced solver reaches on held-out trajectories, against an error bar fixed in advance; the lattice of the Burgers rows and the quadratic tensors of the 3D rows pass it (Appendix A.6).

3.4 Model Architecture

Two requirements shape the model in Figure 1: the head solve needs a usable Jacobian at its stored starting code, and a sparse node set saves time only if the decoder can be evaluated at one node without the others; the learned functions are then also shared by the meshes of a series, while the reduced operators are assembled once per mesh. Widths and training settings are in Appendix C.

Linear skip, for the latent solve. The head is a shallow network plus a linear map,

$$h(z) \;=\; W_{\mathrm{dir}} z \;+\; \varphi_\theta(z), \qquad \varphi_\theta(z) = W_3\,\sigma\big(W_2\,\sigma(W_1 z + b_1) + b_2\big) + b_3, \tag{10}$$

with σ the SiLU activation. Its Jacobian is $J_h(z) = W_{\mathrm{dir}} + J_\varphi(z)$. The first term does not depend on z, so the first Levenberg–Marquardt step from a stored code has a non-zero linear direction even where J_φ is small (this does not guarantee convergence), while φ_θ supplies the curvature of the manifold. The head acts on the R bank coefficients, not on the N nodes, so its cost and its parameter count do not depend on the mesh.

Coordinate-network bank, for node-local evaluation. Each column of the bank is a coordinate network read at the node and multiplied by the boundary factor of (1),

$$G_{x,r} = \mu(x)\, g_{\phi,r}(x), \qquad g_\phi(x) = \mathrm{MLP}_\phi\big(\gamma(x)\big), \quad \gamma(x) = \big[\cos(2\pi\Omega x),\ \sin(2\pi\Omega x)\big], \tag{11}$$

where γ is a random-Fourier-feature embedding (Tancik et al., 2020) whose frequency matrix Ω is initialised at random and then trained for Burgers 2D, Poisson 2D and the L-shape, and kept fixed for the other problems (Appendix C.1). Because g_ϕ reads a coordinate, a row of G is computed from its node alone. A quadrature rule therefore needs only the rows of $\mathcal{S}^+$, and the same trained network is sampled on every mesh.

3.5 Implementation and Training

Online, $J = PKG\,J_h(z)$ with the head Jacobian from forward-mode automatic differentiation in JAX (Bradbury et al., 2018), and each accepted step solves the small damped normal equation directly. Training uses training data only: the coordinate network g_ϕ is fitted by minimising the relative reconstruction error of the training states in its span, with a small conditioning penalty on its columns ((21) and (22) in Appendix C.1), and frozen; the head is then fitted to the exact least-squares bank coefficients of those states, with one stored code per training state (an auto-decoder); last, the bank is ordered once by (2), with one QR and one SVD. At query time the coefficients come from the projected PDE solve, never from the reference. Widths, cohorts and offline costs are in Appendix C.

4 Experimental Setup and Benchmark Problems

Problems and references. The speed case rests on two problems that a coarse mesh does not resolve. (i) *Navier–Stokes 3D, harder family*: incompressible flow on the periodic cube with a quarter of the viscosity of our original family ($\nu \sim \log U(5{\times}10^{-4}, 2.5{\times}10^{-3})$) and four times sharper vortex blobs, horizon $T = 0.2$, reference CNAB2 at 192^3 (Appendix E.1), chosen from five candidate families by criteria fixed before any reduced model was run; evaluated at 64^3, 96^3 and 128^3. (ii) *Unsteady flow past a cylinder* (DFG 2D-2 geometry (Schäfer and Turek, 1996)): vortex shedding on P2/P1 finite elements with 96,781 unknowns, trained at Reynolds numbers 90, 100 and 110, settings frozen at 95, held-out 105 (Appendix E.2). We also report the submission's problems: Burgers 2D/3D, the original Navier–Stokes family, Poisson on an L-shape, and, as correctness checks, Poisson and heat on boxes (Appendix C.3).

Meshes and cohorts are in Table C.8, the sampled families in Table C.9 and the three-dimensional configurations in Table C.13.

Accuracy. We report the worst relative L^2 error over the stated cases and requested times, in percent. Poisson uses the steady reference norm and heat the current reference norm; Burgers and

Table 1: Harder Navier–Stokes family, 128 fresh test cases, three independently trained models (trajectories, per-mesh POD banks and heads re-drawn), settings frozen on 64 development cases. Errors: range over three seeds. Times: seed 0, median over eight cases, online solve only (spread in Table C.7). Speedup over the fastest configuration of a 450-entry solver ladder (any mesh to 128^3, CNAB2 or RK4, any step) at least as accurate on the same cases; range over the two seeds that passed the timing-stability check. *Fast*: fewer steps or tests, also frozen on development cases.

Head on a per-mesh POD bank of width	Worst-case error, three seeds 64^3 / 96^3 / 128^3 (%)	Median error (%)	Time 64^3 / 96^3 / 128^3 (ms)	Speedup over matched solver 64^3 / 96^3 / 128^3
64	1.17–1.36 / 1.45–1.61 / 1.43–1.69	0.34–0.54	16 / 17 / 31	6.9 / 6.4–6.5 / 3.6–3.7×
96	0.58–0.75 / 0.64–0.78 / 0.62–0.82	0.17–0.19	17 / 26 / 43	9.4–11.7 / 7.6–9.5 / 4.7–6.9×
128	0.42–0.60 / 0.43–0.62 / 0.45–0.64	0.10–0.12	31 / 48 / 62	6.3–6.5 / 5.2–7.0 / 4.0–5.3×
128, fast	0.49–0.67 / 0.52–0.68 / 0.47–0.69	0.12–0.14	19 / 31 / 51	9.4–10.4 / 8.0–10.9 / 4.9–6.1×
192	0.28–0.39 / 0.35–0.41 / 0.35–0.42	0.06–0.08	52 / 67 / 121	3.9–5.9 / 5.9–6.2 / 3.3–3.4×
192, fast	0.35–0.48 / 0.39–0.47 / 0.43–0.47	0.09–0.11	25 / 40 / 68	8.0–10.2 / 8.3–10.0 / 5.1×
Linear span, same banks	7.7–70	—	—	0.04–0.56×

Navier–Stokes use the initial-field norm over evolved times; the cylinder uses six fixed acceptance targets (velocity RMS and maximum error, pressure error, Strouhal number, maximum drag and maximum lift). Test cohorts have 16 to 128 cases. In every new experiment (§5.1–5.4) the settings were frozen on development cases before the held-out cases were evaluated; tables on six development cases are labelled as such.

Timing, speedup and matched solvers. Each accuracy–time pair comes from one invocation, with warm-up, synchronisation and medians of repetitions (Appendix C.5). Training, data generation and per-mesh assembly are offline costs (Table C.4). Speedup divides the time of a *matched* solver by the model's: the fastest configuration of a solver ladder whose worst error on the same cases is at or below the setting's own. On the harder Navier–Stokes family the ladder has 450 configurations of a GPU Fourier pseudo-spectral solver over mesh (32^3–128^3), integrator (CNAB2, RK4) and step; on the cylinder, 62 finite-element configurations over mesh, element order, integrator, step and refresh interval; on the original benchmarks, the named solver's own ladder (CG, Crank–Nicolson CG, Newton–BiCGStab, CNAB2), with sine-transform solvers reported where measured. Spread is in Appendix C.4.

5 Numerical Experiments and Results

We ask where the model is faster than a solver of the same accuracy (§5.1–5.3), how it compares with neural-field ROMs and neural operators (§5.4), and what each component contributes (§5.5).

5.1 Interacting Vortices in Three-Dimensional Navier–Stokes

The periodic unit cube contains two interacting vortex blobs, with sharpness $\beta = 8$ and viscosity $\nu \in [5 \times 10^{-4}, 2.5 \times 10^{-3}]$. Relative to the original family, the blobs are narrower and the viscosity is four times lower. The family was selected by resolution and representation criteria fixed before reduced-model runs (Appendix E.1). Figure 2 compares the head and the same bank's span.

In Table 1 the fast width-192 setting is below 0.5 % on every mesh and seed and 5.1–10.2× faster than the matched solver, which needs meshes of 100^3 to 120^3. Across all settings the head is 3.3–11.7× faster than its matched solver. The nonlinear head is what makes this work: the linear span of the same bank is unstable, more so as the bank widens, and slower than the solver. Median errors differ by less than 0.03 percentage points between seeds at widths 96 and above. The speedup narrows with mesh and width (3.3× for width 192 at 128^3). The third seed's timing run failed a timing-stability check twice; its ratios (3.4–10.6×) lie in the same range.

One coordinate network for all meshes. Table 1 uses per-mesh POD banks. On an earlier 32-case cohort one coordinate network fitted once replaces them on all three meshes at 0.90/1.23/1.39 % and 14.2/6.7/3.8×, level with per-mesh POD except at 128^3 (1.17 %, 6.1×); with its 64^3 head reused it gives 1.05/1.06 % at 96^3/128^3 (Table E.1).

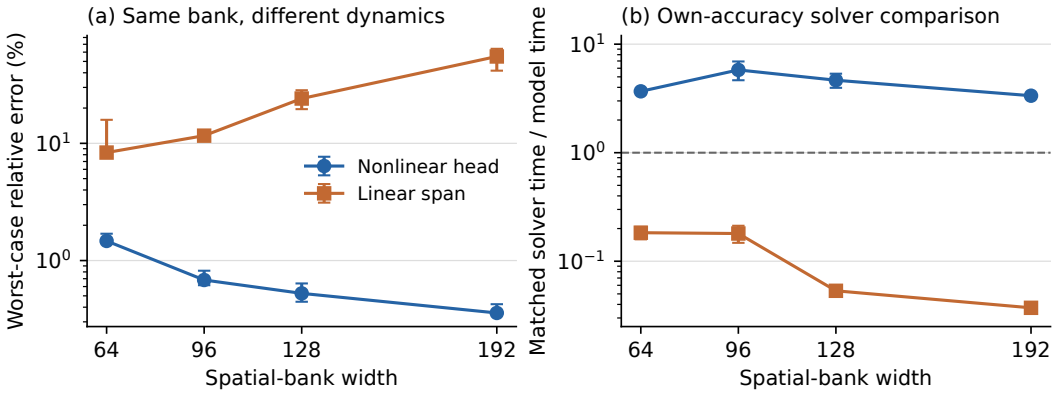


Figure 2: The nonlinear head changes the dynamics within the same bank. Navier–Stokes at 128^3, using the fresh cohort of Table 1. Left: worst-case cohort error, median and range across three independently trained models. Right: speedup over each setting's own accuracy-matched pseudo-spectral solver, median and range over the two models passing the timing gate at every mesh; dashed line: equal cost. These are the accurate head settings, not the fast rows.

Table 2: Cylinder flow, held-out Reynolds number 105. One trained bank at three widths, rollout over 1.7 s (five shedding periods). Comparator: cheapest finite-element run meeting all six targets, screened from a 62-entry ladder on the development case and re-checked at Re 105 (coarser P3/P2 mesh, 16,912 unknowns). Every row meets all six targets. Medians of five repetitions on one 16-thread CPU node (ranges: 37.5–49.4, 1.69–1.72, 0.82–0.83, 0.79 s). *With field output*: velocity rebuilt at all 681 saves.

Setting	Velocity error	Rollout time	Speedup (rollout / with field output)
Cheapest passing finite-element run	9.4×10^{-3}	42.2 s	1×
Width 32, step 1/400	7.0×10^{-3}	1.71 s	24.7× / 11.2×
Width 24, step 1/400	5.5×10^{-3}	0.83 s	51.2× / 15.9×
Width 16, step 1/800	8.3×10^{-3}	0.79 s	53.6× / 17.6×

5.2 Unsteady Flow past a Cylinder

Training and evaluation use a P4/P3 Taylor–Hood reference at step 1/3200; the reduced model is built on a P2/P1 export mesh with 96,781 unknowns. The accuracy-matched finite-element comparator is a separate, cheaper configuration (Table 2). On a non-box geometry one trained bank meets all six targets at every width (Table 2), with a rollout 25–54× faster than the cheapest finite-element run that meets the same targets (11–18× including field output); with a finer step width 32 reaches 1.9×10^{-3}. Over a window three times longer, widths 24 and 32 still meet all six targets (4.0×10^{-3} and 2.4×10^{-3} at step 1/800), while the best of six neural operators trained for this comparison meets three. At Reynolds 80 and 120, outside the band, width 32 meets five and four targets (1.2×10^{-2} against 10^{-2}), narrower widths one to three and the best operator three and four (Table E.2). The bank's boundary factor is a product of distance factors, the inflow is in a lift function, the rollout is Galerkin (Appendix E.2), and a POD basis of equal width performs the same. Replacing the comparator's sparse LU with the tested CPU algebraic-multigrid variants preserves all targets but is slower, so LU remains the stronger measured baseline (Appendix E.2).

5.3 Original Benchmarks against Matched Solvers

Against its own matched solver on the same mesh the fast setting is faster in 27 of the 30 cells of the submission's Table 1 (which include the box rows of Table C.5); it is slower on Burgers 2D at 256^2 and 512^2 (0.58, 0.88×) and Burgers 3D at 64^3 (0.93×), and at Burgers 2D 4096^2 it falls from 27.5 to 13.9× (Figure G.4). On the original Navier–Stokes family a solver free to coarsen its mesh (RK4 on 32^3–48^3) gives the head 0.63/1.65/0.94× at 32^3/64^3/96^3, which is why we moved to the harder family: the advantage appears where a coarse mesh no longer suffices. Transform solvers are faster than our model on box Poisson 2D (0.14–5.05 ms, 1.8–10× faster) and 3D and on heat 3D (fast setting at 0.50–0.61×); our fast setting is 1.1–1.4× ahead on heat 2D; on Burgers 2D a

Table 3: The submission's benchmarks, each setting against the solver matched to *its own* error within that problem's original solver ladder. **Both NM-ROM columns of a row come from one frozen model**: *accurate* is the most accurate tested setting, fixed before the test cases were opened. *Fast*† is the cheapest tested width whose worst error on these test cases is below 5 % (L-shape 5.6 %): **it was selected using the test errors**, so its error is not a held-out estimate. Error is the worst relative L^2 error over the test cases (%). Speedup: matched solver's median time over the NM-ROM's, same GPU allocation, **bold** where the NM-ROM is faster; timing spread in Table C.6. Box Poisson and heat are in Table C.5. Burgers 2D uses a fixed 63×63 lattice for the nonlinear term (§3.3).

		Accurate		Fast†		FOM			Accurate		Fast†		FOM
Problem	Mesh	Err.	Speedup	Err.	Speedup	Err.	Problem	Mesh	Err.	Speedup	Err.	Speedup	Err.
L-shape	256^2	2.02	**32.1**×	5.61	**29.9**×	1.08	Burgers 3D	64^3	1.01	0.21×	3.37	0.93×	0.92
	512^2	2.00	**50.1**×	5.60	**48.7**×	0.74		128^3	1.46	0.57×	4.26	**2.87**×	1.39
	1024^2	1.99	**41.6**×	5.60	**49.4**×	1.77		256^3	2.20	**2.19**×	3.77	**5.60**×	1.52
	2048^2	1.99	**78.4**×	5.60	**97.5**×	1.18	Navier–Stokes	32^3	0.21	0.93×	3.06	**3.56**×	0.19
Burgers 2D	256^2	0.87	0.38×	3.85	0.58×	0.11	(original	64^3	0.21	**2.92**×	3.06	**8.53**×	0.12
	512^2	1.13	0.62×	4.10	0.88×	0.12	family)	96^3	0.21	**8.01**×	3.24	**14.4**×	0.056
	1024^2	1.25	**1.51**×	4.25	**1.70**×	0.13							
	2048^2	1.32	**3.39**×	3.06	**4.43**×	0.13							
	4096^2	1.35	**10.7**×	3.11	**13.9**×	0.14							

Table 4: Burgers 2D at 256^2, 64 held-out test cases, settings of every method frozen on validation data. The CROM- and CoLoRA-style models are our re-implementations. The CoLoRA-style model is given the five parameters that generate the initial condition and solves no equation online; ours takes an initial field. FNO and U-Net are trained on exactly the 4,608 trajectories our head uses, for up to 8.1 hours each (a 4.6-hour run plus one 3.5-hour continuation); most of these runs reached their time budget before our convergence rule, so these measure budget-limited performance, not convergence. The 128-trajectory rows rerun the submission's data budget. Neural operators are compared by prediction error only; inference times are not reported for those rows.

Method	Training trajectories	Worst (%)	Median (%)	ms / query
Ours, linear span, width 384	576 (bank) + 4,608	0.875	0.026	45.2
Ours, nonlinear head, 16 coordinates	4,608	4.10	0.67	32.4
CROM-style, 16 coordinates, accurate / fast setting	same	22.4 / 16.9	0.43 / 1.11	2,818 / 507
CoLoRA-style regression, 16	same	0.240	0.104	49.9
FNO, variant selected on validation (one seed)	4,608	0.48	0.19	—
FNO, paper architecture (three seeds)	4,608	0.43–0.49	0.20–0.23	—
U-Net (three seeds)	4,608	0.45–0.51	0.20–0.22	—
FNO / U-Net, rerun with 128 trajectories	128	9.78 / 5.01	2.61 / 1.23	—
Full-order solver (Newton–BiCGStab)	—	0.109	0.030	17.4

pseudo-spectral solver is faster up to 1024^2 and slower from 2048^2 (development cases). Box Poisson and heat are therefore correctness checks (Appendix C.3).

5.4 Neural-Field ROMs and Neural Operators

Against the CROM-style model, the closest relative (a neural field evolved by the equations online), our head is 4–5.5 times more accurate in the worst case (its median error is higher, 0.67 against 0.43 %) and 16–87 times faster (Table 4): the space/latent factorisation removes repeated differentiation of the spatial network. Against the CoLoRA-style regression the result is a trade-off: our median error is four times lower at slightly lower cost, and its worst-case error is 3.6 times lower. With matched data, our median error is 7–9 times lower than FNO's and U-Net's and below the full-order solver's own median error against the reference; the operators have the lower worst-case error (0.43–0.51 % against 0.875 %). We compare these direct predictors by error, separately from the timing comparison with numerical solvers and ROMs. This replaces the submission's operator claim, which rested on 128 training trajectories (old comparison in Table B.1, whose heat rows, trained for only 6–7 epochs, support no claim). At 1024^2 without retraining our model gives 1.25 %, a zero-shot FNO 2.9–3.2 % and the U-Net about 53 %. On the harder Navier–Stokes family an FNO gives 2.41–2.52 % worst-case error against 0.90–1.39 % for our coordinate bank with the head. On the cylinder's standard window the best operator (an FNO, 180 minutes of final training) also meets all six targets, at 9.9×10^{-3} at Reynolds 105, where our width 32 reaches 1.9×10^{-3} with a fine step and width 24 meets all

six at 5.5×10^{-3}; over the long window and outside the training band no operator meets all six (Appendix B).

5.5 What Each Component Contributes, and Which Knob to Turn

Head against closures. At 16 coordinates on Burgers at 256^2 the head gives 2.56 % against 56.9 % for a linear and 31.8 % for a quadratic closure on the same bank; at 1024^2 the head gives 8.42 % in 35.3 ms, POD-LSPG with 16 modes 41.6 % in 186.5 ms and a quadratic manifold 34.1 % in 758 ms (Table F.3). The fixed lattice takes the query from 1,263 to 49.3 ms at unchanged error. On periodic boxes an interpolated POD basis transfers across meshes as well as the coordinate network, which there adds a single mesh-free object, not accuracy; a tuned POD basis with ECSW matched or beat our bank in an in-sample Burgers study (Appendix F.3).

Which knob. The width R' is the accuracy knob (Figure G.1, Table D.1). The head is preferable when the linear span is inaccurate at affordable width, as on both Navier–Stokes families; the span when a few hundred coefficients are affordable (on Burgers, width 128 reaches 1.10 % in 0.59 of the head's time) and on linear problems.

Limitations. (i) *Out of band.* At cylinder Reynolds numbers 80 and 120 width 32 meets five and four of six targets and narrower widths do not extrapolate. (ii) *Steep data.* On Burgers data steeper than the training range the median error is 5.7 and 16.6 % on two exploratory development regimes; the bank cannot represent those states (Figure G.9). (iii) *Scope.* The Navier–Stokes results cover one harder family and one horizon ($T = 0.2$) at moderate Reynolds numbers; we make no claim about turbulent flow. (iv) *Cylinder.* Its rollout is Galerkin with a lift function, not the solve of §3.2, and POD of equal width performs the same; Neumann and Robin conditions are untested. (v) *Solvers.* On constant-coefficient boxes transform solvers beat our box Poisson and heat-3D models. Tested CPU AMG variants are slower than cylinder LU; GPU sparse-direct and GPU AMG results are not yet available. Mesh-flat reduced evolution does not imply mesh-flat input/output cost (Appendix E.1). (vi) *Selection.* Table 3's fast column was selected on test errors; the quadrature check covers finitely many states and was changed once after seeing results (Appendix A.6).

6 Conclusion and Future Work

One trained NM-ROM gives a ladder of accuracy–cost settings at query time, and its head makes a small model accurate where the linear span of the same bank is not. Against the fastest solver of equal accuracy it is faster where that solver needs a fine mesh and the solution family is low-dimensional, and not where a coarse mesh or a fast transform suffices. Next are wider parameter bands, longer horizons and the least-squares Petrov–Galerkin solve on general geometry.

Ethics Statement

This work develops numerical methods for solving partial differential equations. All data are synthetic, generated by numerical solvers from sampled parameters; no human subjects, personal data or released datasets are involved. We are not aware of harmful uses beyond those of general-purpose simulation software.

Reproducibility Statement

Every table and prose number is taken from versioned run records identifying source, checkpoint, settings, cohort, allocation and audit (Appendix C); failed settings are kept.

AI Use Statement

AI coding agents (Claude, Codex) implemented the methods, baselines and pipelines, proposed and refined experiment designs, analysed and cross-checked results, and drafted and edited text. Every reported number comes from a recorded solver run (Appendix C). The authors reviewed this work and take responsibility for all content.

REFERENCES

K. Azizzadenesheli, N. Kovachki, Z. Li, M. Liu-Schiaffini, J. Kossaifi, and A. Anandkumar. Neural operators for accelerating scientific simulations and design. *Nature Reviews Physics*, 6(5):320–328, 2024.

J. Barnett and C. Farhat. Quadratic approximation manifold for mitigating the Kolmogorov barrier in nonlinear projection-based model order reduction. *Journal of Computational Physics*, 464:111348, 2022.

J. Barnett, C. Farhat, and Y. Maday. Neural-network-augmented projection-based model order reduction for mitigating the Kolmogorov barrier to reducibility. *Journal of Computational Physics*, 492:112420, 2023.

P. Benner, S. Gugercin, and K. Willcox. A survey of projection-based model reduction methods for parametric dynamical systems. *SIAM Review*, 57(4):483–531, 2015.

J. Berman and B. Peherstorfer. CoLoRA: Continuous low-rank adaptation for reduced implicit neural modeling of parameterized partial differential equations. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*, pages 3565–3583, 2024.

J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. JAX: composable transformations of Python+NumPy programs, 2018.

H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han. Once-for-All: Train one network and specialize it for efficient deployment. In *International Conference on Learning Representations*, 2020.

K. Carlberg. Adaptive h-refinement for reduced-order models. *International Journal for Numerical Methods in Engineering*, 102(5):1192–1210, 2015.

K. Carlberg, C. Bou-Mosleh, and C. Farhat. Efficient non-linear model reduction via a least-squares Petrov–Galerkin projection and compressive tensor approximations. *International Journal for Numerical Methods in Engineering*, 86(2):155–181, 2011.

P. Y. Chen, J. Xiang, D. H. Cho, Y. Chang, G. A. Pershing, H. T. Maia, M. M. Chiaramonte, K. T. Carlberg, and E. Grinspun. CROM: Continuous reduced-order modeling of PDEs using implicit neural representations. In *International Conference on Learning Representations*, 2023.

A. Cohen and R. DeVore. Approximation of high-dimensional parametric PDEs. *Acta Numerica*, 24:1–159, 2015.

Devvrit, S. Kudugunta, A. Kusupati, T. Dettmers, K. Chen, I. Dhillon, Y. Tsvetkov, H. Hajishirzi, S. Kakade, A. Farhadi, and P. Jain. MatFormer: Nested transformer for elastic inference. In *Advances in Neural Information Processing Systems*, volume 37, pages 140535–140564, 2024.

A. N. Diaz, Y. Choi, and M. Heinkenschloss. A fast and accurate domain decomposition nonlinear manifold reduced order model. *Computer Methods in Applied Mechanics and Engineering*, 425:116943, 2024.

W. D. Fries, X. He, and Y. Choi. LaSDI: Parametric latent space dynamics identification. *Computer Methods in Applied Mechanics and Engineering*, 399:115436, 2022.

R. Geelen, S. Wright, and K. Willcox. Operator inference for non-intrusive model reduction with quadratic manifolds. *Computer Methods in Applied Mechanics and Engineering*, 403:115717, 2022.

G. H. Golub and V. Pereyra. The differentiation of pseudo-inverses and nonlinear least squares problems whose variables separate. *SIAM Journal on Numerical Analysis*, 10(2):413–432, 1973.

C. Greif and K. Urban. Decay of the Kolmogorov N-width for wave problems. *Applied Mathematics Letters*, 96:216–222, 2019.

Z. Hao, C. Su, S. Liu, J. Berner, C. Ying, H. Su, A. Anandkumar, J. Song, and J. Zhu. DPOT: Auto-regressive denoising operator transformer for large-scale PDE pre-training. In *International Conference on Machine Learning*, pages 17616–17635, 2024.

X. He, Y. Choi, W. D. Fries, J. L. Belof, and J.-S. Chen. gLaSDI: Parametric physics-informed greedy latent space dynamics identification. *Journal of Computational Physics*, 489:112267, 2023.

M. Herde, B. Raonić, T. Rohner, R. Käppeli, R. Molinaro, E. de Bézenac, and S. Mishra. Poseidon: Efficient foundation models for PDEs. In *Advances in Neural Information Processing Systems*, volume 37, pages 72525–72624, 2024.

J. A. Hernández, M. A. Caicedo, and A. Ferrer. Dimensional hyper-reduction of nonlinear finite element models via empirical cubature. *Computer Methods in Applied Mechanics and Engineering*, 313:687–722, 2017.

J. A. Hernández, J. R. Bravo, and S. Ares de Parga. CECM: A continuous empirical cubature method with application to the dimensional hyperreduction of parameterized finite element models. *Computer Methods in Applied Mechanics and Engineering*, 418:116552, 2024.

M. R. Hestenes and E. Stiefel. Methods of conjugate gradients for solving linear systems. *Journal of Research of the National Bureau of Standards*, 49(6):409–436, 1952.

J. S. Hesthaven, B. Peherstorfer, and B. Unger. Nonlinear model reduction for transport-dominated problems. *arXiv preprint arXiv:2602.01397*, 2026.

B. Holzschuh, Q. Liu, G. Kohl, and N. Thuerey. PDE-Transformer: Efficient and versatile transformers for physics simulations. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

M. V. de Hoop, D. Z. Huang, E. Qian, and A. M. Stuart. The cost-accuracy trade-off in operator learning with neural networks. *Journal of Machine Learning*, 1(3):299–341, 2022.

Y. Kim, Y. Choi, D. Widemann, and T. Zohdi. A fast and accurate physics-informed neural network reduced order model with shallow masked autoencoder. *Journal of Computational Physics*, 451:110841, 2022.

M. Kim, T. Wen, K. Lee, and Y. Choi. Physics-informed reduced order model with conditional neural fields. *arXiv preprint arXiv:2412.05233*, 2024.

N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar. Neural operator: learning maps between function spaces. *Journal of Machine Learning Research*, 24(89):1–97, 2023.

A. Kusupati, G. Bhatt, A. Rege, M. Wallingford, A. Sinha, V. Ramanujan, W. Howard-Snyder, K. Chen, S. Kakade, P. Jain, and A. Farhadi. Matryoshka representation learning. In *Advances in Neural Information Processing Systems*, volume 35, pages 30233–30249, 2022.

C. L. Lawson and R. J. Hanson. *Solving Least Squares Problems*. Prentice-Hall, Englewood Cliffs, NJ, 1974.

K. Lee and K. T. Carlberg. Model reduction of dynamical systems on nonlinear manifolds using deep convolutional autoencoders. *Journal of Computational Physics*, 404:108973, 2020.

Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.

L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021.

H. Luo, H. Wu, H. Zhou, L. Xing, Y. Di, J. Wang, and M. Long. Transolver++: An accurate neural solver for PDEs on million-scale geometries. In *International Conference on Machine Learning*, pages 41432–41449, 2025.

L. Magargal, P. Khodabakhshi, S. Rodriguez, J. Jaworski, and J. Michopoulos. Projection-based model-order reduction via graph autoencoders suited for unstructured meshes. *Data-Centric Engineering*, 6:e52, 2025.

D. W. Marquardt. An algorithm for least-squares estimation of nonlinear parameters. *Journal of the Society for Industrial and Applied Mathematics*, 11(2):431–441, 1963.

N. McGreivy and A. Hakim. Weak baselines and reporting biases lead to overoptimism in machine learning for fluid-related partial differential equations. *Nature Machine Intelligence*, 6(10):1256–1269, 2024.

B. Peherstorfer. Breaking the Kolmogorov barrier with nonlinear model reduction. *Notices of the American Mathematical Society*, 69(5):725–733, 2022.

B. Peherstorfer and K. Willcox. Online adaptive model reduction for nonlinear systems via low-rank updates. *SIAM Journal on Scientific Computing*, 37(4):A2123–A2150, 2015.

F. Romor, G. Stabile, and G. Rozza. Non-linear manifold reduced-order models with convolutional autoencoders and reduced over-collocation method. *Journal of Scientific Computing*, 94(3):74, 2023.

F. Romor, G. Stabile, and G. Rozza. Explicable hyper-reduced order models on nonlinearly approximated solution manifolds of compressible and incompressible Navier–Stokes equations. *Journal of Computational Physics*, 524:113729, 2025.

O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241, 2015.

Y. Saad. *Iterative Methods for Sparse Linear Systems*. SIAM, 2nd edition, 2003.

L. Serrano, L. Le Boudec, A. Kassaï Koupaï, T. X. Wang, Y. Yin, J.-N. Vittaut, and P. Gallinari. Operator learning with neural fields: Tackling PDEs on general geometries. In *Advances in Neural Information Processing Systems*, volume 36, pages 70581–70611, 2023.

L. Sirovich. Turbulence and the dynamics of coherent structures, Parts I–III. *Quarterly of Applied Mathematics*, 45(3):561–590, 1987.

R. Ştefănescu and A. Sandu. Efficient approximation of sparse Jacobians for time-implicit reduced order models. *arXiv preprint arXiv:1402.2018*, 2014.

M. Takamoto, T. Praditia, R. Leiteritz, D. MacKinlay, F. Alesiani, D. Pflüger, and M. Niepert. PDEBench: An extensive benchmark for scientific machine learning. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, volume 35, 2022.

M. Tancik, P. P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. T. Barron, and R. Ng. Fourier features let networks learn high frequency functions in low dimensional domains. *Advances in Neural Information Processing Systems*, 33, 2020.

P. Weder, P. Schwerdtner, and B. Peherstorfer. Nonlinear model reduction with Neural Galerkin schemes on quadratic manifolds. *arXiv preprint arXiv:2412.17695*, 2024.

H. Wu, H. Luo, H. Wang, J. Wang, and M. Long. Transolver: A fast transformer solver for PDEs on general geometries. In *International Conference on Machine Learning*, 2024.

M. Yano and A. T. Patera. An LP empirical quadrature procedure for reduced basis treatment of parametrized nonlinear PDEs. *Computer Methods in Applied Mechanics and Engineering*, 344:1104–1123, 2019.

J. Yu, L. Yang, N. Xu, J. Yang, and T. Huang. Slimmable neural networks. In *International Conference on Learning Representations*, 2019.

C. Farhat, P. Avery, T. Chapman, and J. Cortial. Dimensional reduction of nonlinear finite element dynamic models with finite rotations and energy-based mesh sampling and weighting for computational efficiency. *International Journal for Numerical Methods in Engineering*, 98(9):625–662, 2014.

M. Schäfer and S. Turek. Benchmark computations of laminar flow around a cylinder. In *Flow Simulation with High-Performance Computers II*, volume 52 of *Notes on Numerical Fluid Mechanics*, pages 547–566. Vieweg, 1996.

A METHOD DETAILS

This appendix gives the details of §3 that the main text leaves out: the ordering of the bank, the row scaling of each PDE, the Burgers stencil, the Navier–Stokes family, the quadrature fit and the solver constants. On the square the tests are eigenvectors of the Laplacian, $PK = \Lambda P$ with Λ diagonal, and every PDE scales the tested residual PR by the inverse of a diagonal matrix built from Λ. On the unit cube the tests are the M lowest discrete sine modes $\sqrt{8}\prod_i \sin(\pi k_i x_i)$, ordered by $|k|^2$ ($M = 512$ for Poisson, 1024 for heat and about $4R'$ for Burgers, completed to a full shell of equal $|k|^2$), and the boundary factor is $\mu(x) = 64\prod_i x_i(1 - x_i)$. On the L-shaped domain the tests are the $M = 257$ lowest eigenvectors of its five-point matrix, computed once per mesh by shift-invert Lanczos, and the boundary factor is $\mu(x) = 4\min\big(x_1,\, x_2,\, r,\, t,\, \|x - (\frac{1}{2}, \frac{1}{2})\|\big)$, with $r = 1 - x_1$ for $x_2 \le \frac{1}{2}$ and $\frac{1}{2} - x_1$ otherwise, and $t = 1 - x_2$ for $x_1 \le \frac{1}{2}$ and $\frac{1}{2} - x_2$ otherwise.

A.1 ORDERING THE BANK

The ordering is a principal component analysis of the training fields, done inside the space the bank spans. First we make the bank orthonormal with a thin QR factorisation, $G = Q_G R_G$, so that a field Ga has coordinates $R_G a$ in the orthonormal basis Q_G. For every training field u_i we take these coordinates and divide by $\|u_i\|$, so that every field counts equally, and stack the results as the rows of a matrix $\mathcal{A}$. Its singular value decomposition $\mathcal{A} = USV^\top$ gives the directions the training fields use, in the columns of V, sorted by the singular values in S. We set $T = R_G^{-1}V$, so that $\widehat{G} = GT = Q_G V$: its columns are orthonormal and sorted by importance, and by the Eckart–Young theorem its first R' columns are the best R'-dimensional subspace of the bank for the scaled training fields. Because T is invertible, the full-width bank gives the same trial states as before (to 10^{-10} in our runs), and the head is kept unchanged. The step uses training data only and has no trainable weights.

A.2 POISSON

The tested residual is scaled by Λ^{-1}. Because the retained tests are orthonormal and $Ku^\star = F$, the scaled residual $\Lambda^{-1}P(K\tilde{u} - F) = P(\tilde{u} - u^\star)$ is exactly the tested error, so the objective is the squared discrete L^2 error of the reduced state in the retained modes. In the span, the thin QR factorisation of $P\widehat{G}_{R'}$ is computed once per mesh and width, and a query is one matrix–vector product and one triangular solve. At $R' = R$ the span solve reaches the bank floor in the tested norm.

In the span,

$$P\,R\big(\widehat{G}_{R'}c\big) = \big(PK\widehat{G}_{R'}\big)\,c - PF, \tag{12}$$

so a query is one least-squares solve with a fixed $M \times R'$ matrix, factorised once per mesh and width. With the head, (5) iterates in z with $J = PKG\,J_h(z)$.

A.3 HEAT

The semi-discretisation $du/dt = -\kappa K u$, with diffusivity κ, is advanced by Crank–Nicolson with the residual

$$R(u_{n+1}) = \big(I + \tfrac{\Delta t\kappa}{2}K\big)u_{n+1} - \big(I - \tfrac{\Delta t\kappa}{2}K\big)u_n. \tag{13}$$

The tested Crank–Nicolson residual (13) is scaled by $(I + \frac{\Delta t\kappa}{2}\Lambda)^{-1}$, which turns each step into $P\tilde{u}_{n+1} = D\,P\tilde{u}_n$ with $D = \operatorname{diag}\big((1 - \frac{\Delta t\kappa}{2}\lambda_i)/(1 + \frac{\Delta t\kappa}{2}\lambda_i)\big)$: each tested mode is multiplied by its Crank–Nicolson factor. In the span the step is the linear recurrence

$$c_{n+1} = B_{R'}^{+} D\, B_{R'}\, c_n, \tag{14}$$

with $B_{R'} = P\widehat{G}_{R'}$ and c_0 fitted to the initial field. Because the step matrix $B_{R'}^{+} D\, B_{R'}$ is fixed, its powers at the output times are formed once offline, so a query fits c_0, applies one small matrix per output time and rebuilds the fields. This is exactly the per-step least-squares solution of the tested Crank–Nicolson residual at the full-order time step $\Delta t = 0.025$, with no further time-discretisation error, and every heat result in the paper is computed this way.

A.4 BURGERS

For $u_t + u(u_x + u_y) = \nu\Delta u$, with viscosity ν, backward Euler gives the residual

$$R(u_{n+1}) = u_{n+1} - u_n + \Delta t\big(A(u_{n+1}) + \nu K u_{n+1}\big), \tag{15}$$

where $A(u)$ is the advection term. The full-order problem uses the sign-upwind stencil

$$A(u)_{ij} \;=\; u_{ij}\left(\delta_x u + \delta_y u\right)_{ij}, \qquad (\delta_x u)_{ij} = \frac{1}{\Delta x}\begin{cases} u_{ij} - u_{i-1,j}, & u_{ij} > 0, \\ u_{i+1,j} - u_{ij}, & u_{ij} \le 0, \end{cases} \tag{16}$$

with δ_y analogous, zero ghost values on the walls and backward Euler in time. The tested residual (15) is scaled by $(I + \Delta t\,\nu\Lambda)^{-1}$, the modal form of the Helmholtz preconditioner the full-order Newton solver uses. Each step is warm-started from the previous coefficients, and a linear or quadratic extrapolation is used when its residual is smaller. At every mesh the step assembles the Jacobian analytically, clips the step, solves the damped normal equation by Cholesky factorisation and carries the damping between steps.

A.5 NAVIER–STOKES

For $u_t + (u\cdot\nabla)u = -\nabla p + \nu\Delta u$, $\nabla\cdot u = 0$, the implicit midpoint rule gives the residual

$$R(u_{n+1}) = u_{n+1} - u_n + \Delta t\big((\bar u\cdot\nabla)\bar u + \nabla\bar p - \nu\Delta\bar u\big), \qquad \bar u = \tfrac{1}{2}(u_n + u_{n+1}). \tag{17}$$

The unknown is the three-component, zero-mean velocity on the periodic unit cube; the sampled family is in Table C.9. The data and the full-order comparator come from CNAB2 with two-thirds dealiasing. The bank is the rank-64 POD basis of 768 training states (six frames of each of 128 trajectories) generated on the query mesh, each centred on its own energy centroid. It is ordered once by the eigenvectors of the second moment of the training coefficients, which gives the nested widths $R' \in \{64, 48, 32, 16, 8\}$; the centroid moves with the vortices, and a query solves for its three-component shift at every step together with the latent coordinates. All linear reduced operators are computed offline, and the advection term is evaluated with a precomputed quadratic tensor (§3.3). The head is trained as an auto-decoder on the centred bank coefficients of training states. Bank and head are built once per mesh (32^3, 64^3, 96^3) and frozen. A coordinate network fitted once for all three meshes gives 0.18–0.20 % with the head on this family (per-mesh POD 0.15 %); the harder family is in Appendix E.1. The tests are the $M = 292$ lowest divergence-free Fourier modes. A query centres the initial field, projects it on the bank and, for the head, starts from a stored training code. It then takes 10 implicit-midpoint steps of $\Delta t = 0.02$, each solved by three damped Gauss–Newton sweeps with an analytic Jacobian, starting from a linear extrapolation of the last two steps. With the centring held fixed the same bank misses every case by far more than 5 %, so the centring is necessary.

A.6 EMPIRICAL QUADRATURE

With $g_i^{(\ell)} = P_{:,i}A_i(\tilde u^{(\ell)}) \in \mathbb{R}^M$ the per-node contribution at a stored state $\tilde u^{(\ell)}$ that the solver reaches on training trajectories, the constraint at that state is $\sum_i w_i\, g_i^{(\ell)} = PA(\tilde u^{(\ell)})$. Stacking the constraints gives a matrix E and a vector b, and we solve

$$\min_{w\ge 0} \|E\,w - b\|_2 \tag{18}$$

by NNLS (Lawson and Hanson, 1974), which returns a sparse w with support $\mathcal{S}$ of size $N_{\mathrm{eq}} \ll N$. Computing A_i needs $\tilde u$ at node i and its stencil neighbours, so $|\mathcal{S}^+| \le 5N_{\mathrm{eq}}$ in two dimensions and $7N_{\mathrm{eq}}$ in three. A convolutional or attention decoder would couple its output nodes, so one node would cost a pass over the whole grid; this is why the bank is a coordinate network.

On one state, a rule's error is

$$\rho(u) = \frac{\|\sum_{i\in\mathcal{S}} w_i P_{:,i}A_i(u) - PA(u)\|_2}{\|PA(u)\|_2}, \qquad \rho_{\max} \le 0.116, \tag{19}$$

over reachable states. The bar was fixed before any check was run. A rule is confirmed when $\rho_{\max}$ stays below it first on one set of trajectories disjoint from the fit and the evaluation cases, then on

a further set checked last. Backward Euler never evaluates advection at the initial state. The first registered version of the check included that state; we removed it after observing that it was the only state at which the widest span settings failed, and this is the only change we made to the check after seeing results. The fit (18) normalises each row, grows the support greedily by the Lawson–Hanson criterion with an exact non-negative refit after each addition, and stops at N_{eq} nodes. The fit states are states the dense solver reaches on fit trajectories; the test states come from trajectories disjoint from the fit and the evaluation cases. A deployed rule is its node indices, weights and the cached $N_{\mathrm{eq}} \times 5 \times R$ bank block. Correction: the Burgers rows of Table 3 do *not* use a fitted rule. They use a rule that needs no fit: the uniform interior lattice with 64 intervals per axis, $63 \times 63 = 3969$ nodes with equal weights $(1/64)^2$, checked on reached states by (19). At 1024^2 with $R' = 384$ it gives the dense-residual error (0.2108 %) in 49.3 instead of 1,263 ms; sparser lattices (961 or 1,953 nodes) failed or did not confirm the check at $R' = 384$.

A.7 Solver constants

With r_w the scaled tested residual and J its Jacobian, the stopping test is scale free, $\|J^\top r_w\|_2 \le \eta_{\mathrm{tol}} \|J\|_F \, \|r_w\|_2$, so one tolerance serves every mesh and width. The Poisson, L-shape and heat settings of Table 3 are span solves with no stopping test. Two-dimensional Burgers uses $\eta_{\mathrm{tol}} = 10^{-2}$ at 256^2–1024^2 and 10^{-3} at 2048^2 and 4096^2, with at most 600 attempts per step. Three-dimensional Burgers solves the first three steps to $\eta_{\mathrm{tol}} = 10^{-3}$ (at most 50 attempts) and then takes one sweep per step. Navier–Stokes takes a fixed three Gauss–Newton sweeps per step with fixed damping $\lambda = 10^{-6}$. Damping starts at $\lambda_0 = 10^{-6}$ (Poisson, Burgers in two and three dimensions) or 10^{-4} (heat), is carried between time steps, is divided by 3 on acceptance (down to 10^{-12}) and multiplied by 10 on rejection (up to 10^{14}). The *bank floor* is the projection error onto the span of the bank, which no solve can beat.

B Neural operators

Table B.1: Historical accuracy-only operator comparison. Burgers operators used 128 trajectories and are superseded by the matched-data Table 4; heat runs completed only 6–7 epochs and support no claim. Worst-case relative errors (%) at each measured cell; both NM-ROM settings come from one trained model. Operator sizes were selected on validation data under the reported training budget. No operator timing comparison is made.

PDE	N	NM-ROM accurate	NM-ROM second setting	FNO	U-Net	Transolver	DeepONet
Burgers 2D	256^2	—	4.10	8.76	5.98	6.83	64.2
	1024^2	1.25	8.42	14.8	11.5	14.9	68.2
	2048^2	1.32	9.04	18.0	22.6	33.8	67.1
Heat 2D	1024^2	0.13	—	3.59	1.56	2.57	4.60
	2048^2	0.13	—	3.98	2.76	3.66	10.9
Navier–Stokes 3D	32^3	0.21	3.24	0.53	0.68	0.78	51.9
	64^3	0.21	3.24	1.76	1.58	1.80	47.6
	96^3	0.21	3.24	1.51	2.29	1.16	51.4

We trained an FNO (Li et al., 2021), a U-Net (Ronneberger et al., 2015), a DeepONet (Lu et al., 2021) and a Transolver (Wu et al., 2024) on Burgers 2D, heat 2D and Navier–Stokes 3D, each at the mesh it is evaluated on, with AdamW, one fixed seed per network (Table B.4) and a 3000 s wall budget per network, and scored them by the same metric as our model. They produce the requested output times in one pass, so their output times are fixed at training.

Cells of Table B.1. Error is over evolved times for Burgers and all times for heat. Cells: Burgers 256^2, 1024^2 and 2048^2 (A100, A100 and H100 GPUs), the 64 test cases of Table 3, FOM Newton–BiCGStab tol 3×10^{-3}, Δt=0.005 (0.11, 0.13 and 0.14 %); heat 1024^2 and 2048^2 (A100 and H200 GPUs), the 16 test cases of Table 3, FOM CN–CG Δt=0.025, rtol 10^{-4} (0.045 %); Navier–Stokes 32^3 and 64^3, the 32 test cases of Table 3, FOM CNAB2 with 40 and 50 steps (0.19 and 0.12 %); Navier–Stokes 96^3, 32 test cases, FOM CNAB2 with 70 steps (0.056 %). The accurate NM-ROM

setting is the span with $R' = 384$; the second setting uses the head with $k = 16$ on Burgers and the span with $R' = 16$ on Navier–Stokes. Blank cells indicate unavailable measurements.

Table B.2: Best NM-ROM against other nonlinear-manifold and projection ROMs on Burgers 2D at matched latent dimension ($k = 16$ for every baseline and for our head). Error is the worst evolved same-grid relative L^2 error (%). Speedup uses one full-order setting per cell, the fastest at least as accurate as the NM-ROM accurate setting, same allocation; *italic* below 1×; **bold** marks an NM-ROM setting more accurate and faster than every baseline. At 256^2 and 512^2 the cohort is 32 validation cases (never used for any choice) and every method, ours included, uses a dense residual, so no times are given; at 1024^2 and 2048^2 it is the 64 held-out test cases of Table 3. The shallow masked-autoencoder NM-LSPG of Y. Kim et al. (2022) was checked against its published result before use (Table C.12); its hyper-reduction was not reproduced and it was not run from 1024^2. The quadratic manifold (Geelen et al., 2022) was run at 1024^2 and 2048^2 only. Every latent size and the memory use are in Tables C.12 and B.3. The reduced-model timing comparison uses a separate allocation.

		NM-ROM accurate		NM-ROM head, k=16		Kim et al. NM-LSPG		POD-LSPG		Quadratic manifold	
PDE	N	Err. (%)	Speedup	Err. (%)	Speedup	Err. (%)	Speedup	Err. (%)	Speedup	Err. (%)	Speedup
Burgers 2D	256^2	—	—	6.79	—	145.40	—	41.53	—	—	—
	512^2	—	—	7.72	—	174.99	—	42.01	—	—	—
	1024^2	1.25	*0.24*×	**8.42**	**2.17**×	—	—	41.6	*0.41*×	34.1	*0.101*×
	2048^2	1.32	*0.68*×	**9.04**	**4.79**×	—	—	41.7	*0.43*×	34.2	*0.096*×

Other reduced-order models at 1024^2. Table B.3 gives POD-LSPG and a quadratic manifold (Geelen et al., 2022) at several sizes on the six development cases, in a separate job from Table B.1. Neither is both more accurate and faster than our setting. POD-LSPG needs 256 or more unknowns to approach 1 %; the quadratic manifold stays above 7 % up to 64.

Table B.3: Development table. Burgers 2D at 1024^2, one allocation (A100 80 GB, six development cases): worst and median evolved same-grid error, median GPU query time, and speedup over one full-order setting chosen on these development cases (Newton–BiCGStab, tol 3×10^{-3}, Δt=0.005, 70.5 ms, the fastest setting at least as accurate as the NM-ROM accurate setting); below 1× the row is slower. FOM rows are Newton–BiCGStab settings; 1023^2 is the number of interior nodes.

Method	Unknowns	Worst (%)	Median (%)	GPU ms	Speedup
NM-ROM, span (R'=384)	384	0.21	0.065	318	0.22×
NM-ROM, head (k=16)	16	2.29	1.04	34.9	2.02×
POD-LSPG	16	29.3	22.1	178	0.40×
POD-LSPG	64	7.41	3.66	911	0.077×
POD-LSPG	256	1.10	0.42	12147	0.006×
POD-LSPG	512	0.44	0.053	47503	0.001×
Quadratic manifold	16	22.6	15.0	733	0.096×
Quadratic manifold	32	13.0	6.68	2547	0.028×
Quadratic manifold	64	7.26	3.58	8257	0.009×
FOM, tol 10^{-2}, Δt=0.01	1023^2	1.66	1.20	37.0	—
FOM, tol 3×10^{-3}, Δt=0.005	1023^2	0.048	0.038	70.5	—

Navier–Stokes cells. Both sides draw from the same 512 training trajectories. The operators train on 448 and choose their size on the other 64, the same split the head used. At 32^3 CNAB2 with 40 steps reaches 0.185 % on the test cases, below the accurate setting's 0.206 %, so the rule selects 40 steps.

How much data each side was given. On heat and Navier–Stokes both sides are trained on the same trajectories. On Burgers the operators of Table B.1 use 128 trajectories(the matched-data comparison is Table 4), while our bank is fitted on 576 and the head on 4608 of the same family (Table C.1). Both sides draw initial conditions from the same family and are scored at the same output times. At 256^2 each operator target is a 4096-interval solution restricted to the 256-interval grid. At 1024^2 and 2048^2 the trajectories are recomputed on the evaluation mesh with a tight Newton tolerance, so there the operators are trained on exactly the fields they are scored against, while our

model, trained at 256^2, is used unchanged. An operator sees one input–output pair per trajectory, while the head is fitted per state.

Table B.4: Training of the neural operators of Table B.1, one row per trained network shown there: parameters, epochs completed within the 3000 s wall budget, and why training stopped. Every network used AdamW, weight decay 10^{-4}, batch 8 (by gradient accumulation where memory required it), seed 20260914 and the checkpoint of its best validation epoch. Training sets: the training trajectories and split of our model for each problem (heat 512 with 16 validation, Navier–Stokes 448 with 64 validation). The Burgers 2048^2 FNO was stopped by host memory after 19 epochs.

Cell	Operator	Parameters	Epochs	Stopped by
Burgers 2D, 256^2	FNO	17,877,317	692	wall budget
	U-Net	17,462,021	908	wall budget
	Transolver	3,108,240	1628	wall budget
	DeepONet	2,569,349	537	early stopping
Burgers 2D, 1024^2	FNO	17,877,317	37	wall budget
	U-Net	7,763,461	223	wall budget
	Transolver	3,108,240	131	wall budget
	DeepONet	2,569,349	308	wall budget
Burgers 2D, 2048^2	FNO	17,877,317	19	host memory
	U-Net	7,763,461	62	wall budget
	Transolver	3,108,240	24	wall budget
	DeepONet	2,569,349	69	wall budget
Heat 2D, 1024^2	FNO	17,877,189	7	wall budget
	U-Net	7,763,173	71	wall budget
	Transolver	12,746,048	228	wall budget
	DeepONet	2,568,917	98	wall budget
Heat 2D, 2048^2	FNO	17,877,189	6	wall budget
	U-Net	7,763,173	23	wall budget
	Transolver	14,126,144	394	wall budget
	DeepONet	2,568,917	31	wall budget
Navier–Stokes 3D, 32^3	FNO	18,888,495	624	wall budget
	U-Net	22,581,327	458	wall budget
	Transolver	6,003,816	269	wall budget
	DeepONet	5,741,519	298	early stopping
Navier–Stokes 3D, 64^3	FNO	18,888,495	88	wall budget
	U-Net	22,581,327	80	wall budget
	Transolver	17,996,928	110	wall budget
	DeepONet	5,741,519	232	wall budget
Navier–Stokes 3D, 96^3	FNO	18,888,495	110	wall budget
	U-Net	22,581,327	62	wall budget
	Transolver	6,619,288	510	wall budget
	DeepONet	5,741,519	70	wall budget

C Experimental configuration and reproducibility

Tables C.8 and C.9 identify the two-dimensional studies and Table C.13 the three-dimensional ones. Within each study, training precedes evaluation, and the selected bank, its ordering and the head remain frozen. Deployment changes reduced coordinates, not network weights. Table C.1 gives the training configuration of every frozen model and Table C.2 its offline cost.

C.1 Bank and head training losses

The bank minimises the relative reconstruction error of the training states in its span, with a small conditioning penalty on its columns. With u_i, $i = 1, \ldots, S$, the training states of a batch, evaluated on a random batch of P grid points, $\widehat{G}_\phi$ the bank columns on those points scaled by their training

Table C.1: Training configurations of the frozen checkpoints, transcribed from the committed configuration and code files of each study.

Model	Stage	Network	Optimiser, learning rate	Steps	Batch	Training data
Burgers 2D ($k = 16$, $R = 512$)	bank	128 Fourier features (scale 4), width 1024, 2 layers	AdamW, wd 10^{-5}, warm-up + cosine $10^{-3} \to 10^{-5}$	300000	all states $\times$ 4096 points	16384 states of 576 trajectories
	head	width 512, 2 layers	Adam, warm-up + cosine $10^{-3} \to 10^{-5}$	200000	4096 states	131072 states of 4608 trajectories
Poisson 2D ($k = 32$, $R = 512$)	bank	64 Fourier features (scale 4), width R, 2 layers	Adam, four phases at 10^{-3}, 3×10^{-4}, 3×10^{-4}, 10^{-4}	100000, 40000, 30000, 30000	64 sources $\times$ 4096 points	2611 fit / 461 validation sources
	head	width 128, 2 layers	Adam, 10^{-3}	150000 (full batch)	all	as bank
L-shape ($k = 16$, $R = 512$)	bank / head	64 Fourier features (scale 4), 2 layers; head width 128, 2 layers	as Poisson $k = 32$	100000, 40000, 30000, 30000; head 150000	64 sources $\times$ 4096 points	2611 fit / 461 validation sources
Poisson 3D ($k = 16$, $R = 128$)	bank / head	64 Fourier features (scale 1.5), widths 256; head width 256 + skip	Adam, cosine (floor 0.03); 10^{-3} bank, 5×10^{-4} head	150000 / 100000	64 states $\times$ 2048 points / 128 states	512 training / 16 validation fields
Heat 2D ($k = 8$, $R = 128$)	bank / head	32 Fourier features (scale 1.5), width 256; head width 256	Adam, clip 1, exact coefficient refit every 10000 steps; 10^{-3} bank, 5×10^{-4} head	80000 / 80000	64 states $\times$ 2048 points / 128 states	512 training / 16 validation draws at 64^2
Heat 3D ($k = 32$, $R = 320$)	bank / head	128 Fourier features (scale 2), width 512, 3 layers; head width 512, 2 layers + skip	bank: Adam, clip 1, warm-up + cosine $10^{-3} \to 10^{-5}$; head: Adam, cosine from 5×10^{-4} (floor 0.03)	40000 (step 32000 kept) / 120000	512 states on the full 32^3 grid / 256 states	2048 training / 256 validation trajectories (6 times each)
Burgers 3D ($R = 512$, span only)	bank	128 Fourier features (scale 2), width 1024, 3 layers	Adam, clip 1, warm-up + cosine $10^{-3} \to 10^{-5}$	12000 (step 7000 kept)	128 states $\times$ 32768 points per mesh	18432 states of 1536 trajectories at each of 32^3, 64^3, 128^3; 96 validation trajectories
Fourier features ((11)): frequencies trained for Burgers 2D, Poisson 2D and the L-shape, fixed for the others (Table C.3).						
Navier–Stokes 3D ($k = 8$, $R = 64$)	bank	POD of centred snapshots, built per mesh (no trained weights)	—	—	—	768 centred states of 128 trajectories
	head	width 256, 2 layers + skip, one per mesh	Adam, clip 1, warm-up + cosine $10^{-3} \to 10^{-5}$	100000 (full batch)	all	9408 training / 1344 validation states of 512 trajectories

Table C.2: Offline cost of each deployed model, in seconds of wall-clock time on one A100 or H200 GPU (the heat orderings on a workstation GPU). Setup is the per-mesh assembly of the tests and reduced operators, over the range of meshes. Dashes mark costs not recorded separately.

Problem	Bank	Head	Ordering	Setup per mesh
Poisson 2D	434	141	4.0	33.6 at 4096^2
Poisson, L-shape	375	98	79	869 at 2048^2 (eigenvectors 830)
Poisson 3D	101	54	4.5	10.7–48.0
Heat 2D	56	1113	14	16–260
Heat 3D	808	94	96	6.5–333
Burgers 2D	10,685	339	34	—
Burgers 3D	2072	—	—	14–279
Navier–Stokes 3D	—	95	—	—

RMS, and

$$e_i = \frac{\|G_\phi c_i - u_i\|_2^2}{\|u_i\|_2^2}, \tag{20}$$

two forms were used. *Joint form* (Poisson 2D and 3D, the L-shape, heat 2D, Burgers 2D): the coefficients c_i are trained together with the network,

$$\mathcal{L}_{\text{joint}}(\phi, \{c_i\}) = \frac{1}{S}\sum_{i=1}^{S} e_i + \lambda \text{ mean}\left[\left(\widehat{G}_\phi^\top \widehat{G}_\phi / P - I\right)^2\right] + \gamma \frac{1}{S}\sum_{i=1}^{S} e_i^2, \tag{21}$$

where the mean is over the R^2 entries. The c_i are free vectors for Poisson 3D and heat 2D (heat 2D resets them to the exact least-squares solution every 10,000 steps); for Burgers 2D they are head outputs $c_i = h(z_i)$ with free latent codes z_i, and $\|u_i\|^2$ is replaced by a global mean square; Poisson 2D and the L-shape alternate head-output phases with a free-coefficient phase started from the exact least-squares solution. *Variable-projection form* (heat 3D, Burgers 3D): the coefficients are eliminated exactly at every step, $c_i = G_\phi^+ u_i$ through a differentiable thin QR factorisation (Golub and Pereyra, 1973), so e_i is the projection error itself, and

$$\mathcal{L}_{\text{VP}}(\phi) = \log \mathcal{E}_{\text{mean}} + \beta \log\left(\frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}} e_i^p\right)^{1/p} + \lambda_c \text{ mean}\left[(C_\phi - I)^2\right], \qquad p = 4, \tag{22}$$

where $\mathcal{B}$ is the snapshot minibatch, the power mean penalises its largest errors, and C_ϕ is the matrix of cosines between bank columns. For Burgers 3D, $\mathcal{E}_{\text{mean}} = S^{-1}\sum_i e_i$ and the first two terms are summed over the three training meshes. For heat 3D, the mean term is evaluated through the leading POD modes of the unit-normalised training states, not a snapshot minibatch mean. If v_ℓ are those orthonormal modes, λ_ℓ their snapshot-Gram eigenvalues, S the full training state count, and Q the thin-QR basis of the current bank, the implemented term is

$$\mathcal{E}_{\text{mean}} = \sum_{\ell=1}^{1536} \frac{\lambda_\ell}{S}\|(I - QQ^\top)v_\ell\|_2^2.$$

This retains the leading-mode contribution to the full training mean; the omitted POD tail is recorded in the training output. The power-mean term still uses a random minibatch of normalised snapshots. Once the bank is frozen, the coefficients used to order it and as head-training targets are the exact least-squares solutions $c_i^\star = G^+ u_i$, and the head (with one free code z_i per state) minimises the relative field error

$$\mathcal{L}_{\text{head}}(\theta, \{z_i\}) = \frac{1}{S}\sum_{i=1}^{S}\varepsilon_i + \gamma_h \frac{1}{S}\sum_{i=1}^{S}\varepsilon_i^2, \qquad \varepsilon_i = \frac{\|G\,(h(z_i) - c_i^\star)\|_2}{\|u_i\|_2}, \tag{23}$$

with $\gamma_h = 0.1$ for heat and the three-dimensional problems and 0 otherwise. At query time the coefficients come from the projected PDE solve. The weights per problem are in Table C.3.

Table C.3: Bank-loss form and weights per problem, read from the training code of each frozen model. $P \times S$: grid points per batch $\times$ states per batch. Ω: Fourier-feature frequencies of (11). Navier–Stokes banks are POD bases (no loss); the cylinder bank is described in Appendix E.2.

Problem	Form	Penalty weight	$P \times S$	γ / β	Coefficient handling	Ω
Poisson 2D	(21)	$\lambda = 10^{-4}$	4096×64	$\gamma = 0$	head phases + free phase	trained
L-shape	(21)	$\lambda = 10^{-4}$	4096×64	$\gamma = 0$	as Poisson 2D	trained
Poisson 3D	(21)	$\lambda = 10^{-6}$	2048×64	$\gamma = 0$	free vectors	fixed
Heat 2D	(21)	$\lambda = 10^{-6}$	2048×64	$\gamma = 0.1$	free, exact reset every 10^4 steps	fixed
Burgers 2D	(21)	$\lambda = 10^{-4}$	$4096\times$ all states	$\gamma = 0$	head outputs, free codes; global norm	trained
Heat 3D	(22)	$\lambda_c = 10^{-3}$	full 31^3 grid $\times 512$	$\beta = 0.5$	eliminated (thin QR)	fixed
Burgers 3D	(22)	$\lambda_c = 10^{-3}$	32768×128 per mesh	$\beta = 0.5$	eliminated, three meshes	fixed

C.2 Total cost and break-even

Table C.4 gives the number of queries needed to repay the recorded offline cost (training and setup, and data generation where recorded), each fast setting against its own accuracy-matched solver at the

largest mesh. Offline training and setup are an upfront investment in repeated queries. The relevant comparison is total cost at the intended query count, with each measured saving tied to its comparator and output scope; a positive online speedup alone does not establish amortisation. Cylinder: training trajectories at Reynolds 90/100/110 with the high-order reference solver (66,400 s), bank training (4,100 s), export and reduced operators (about 440 s); generating the training data with the cheapest passing finite-element setting would lower this. Navier–Stokes: training trajectories 0.45, bank 0.34, head data 1.8, head fits 0.1, assembly and selection on three meshes 0.6 GPU-hours, which together cover all three meshes (33,000–53,000 queries at 64^3 or 96^3). Box Poisson never breaks even against the sine transform. Poisson and heat offline costs exclude data generation, which was not recorded; the per-problem components are in Table C.2. Figure G.10 draws the cumulative cost at 1024^2.

Table C.4: Offline cost, saving per query and break-even, each fast setting against its own accuracy-matched solver at the largest mesh.

Problem	Offline cost	Saved per query	Break-even (queries)
Poisson 4096^2, against conjugate gradients	613 s	3.85 s	159
Heat 2D 4096^2, against CN–CG	1,443 s	1.02 s	1,417
(against the modal solver, $R' = 48$ arm)			about 370,000
Cylinder, held-out Reynolds 105 (width 32)	about 71,000 node-s	40.5 s	about 1,800
Burgers 2D 4096^2, including data generation	12,043 s	0.60 s	about 20,000
Harder Navier–Stokes 128^3, fast width 192	about 3.3 GPU-h	0.28 s	about 43,000

C.3 Box problems as correctness checks

Poisson and heat on the square and cube have exact transform solvers. Table C.5 gives the submission's rows with the fast setting against its own matched solver and the ratio of a transform solver's time to the fast setting's (below 1: the transform solver is faster). The accurate error is the bank's projection floor at every mesh, which checks that the reduced solve reaches the best fit in the span.

Table C.5: Box Poisson and heat (test cases). Accurate: against the submission's named iterative solver (CG, CN–CG). Fast†: test-selected width (as in Table 3) against its own matched rung of the same solver. Transform: sine-transform (Poisson) or modal (heat) solver time over the fast setting's, timed in a separate job on development or held-out cases; * the arm timed there is $R' = 64$ (Poisson 3D), $R' = 48$ (heat 2D) or $R' = 128$ (heat 3D 64^3–256^3) rather than the arm of this table. ‡ timing gate failed (order effect up to 1.30); speedup indicative only.

Problem	Mesh	Acc. err. (%)	Acc. speedup	Fast err. (%)	Fast speedup (matched)	FOM err. (%)	Transform / fast
Poisson 2D	256^2	0.84	18.2×‡	4.07	27.0×‡	0.77	0.57
	1024^2	0.83	31.1×	4.06	70.9×	0.33	0.33
	2048^2	0.83	64.4×	4.06	174.5×	0.22	0.36
	4096^2	0.83	108×	4.06	319×	0.58	0.44
Poisson 3D	32^3	0.23	4.42×	2.01	4.45×	0.16	0.59*
	64^3	0.23	8.07×	1.98	8.23×	0.11	0.52*
	128^3	0.23	13.6×	1.97	19.9×	0.075	0.36*
	256^3	0.23	19.3×	1.96	40.0×	0.17	0.46*
Heat 2D	1024^2	0.13	105×	1.08	83.0×	0.045	1.11*
	2048^2	0.13	216×	1.08	213×	0.045	1.20*
	4096^2	0.13	231×	1.08	258×	0.054	1.41*
Heat 3D	32^3	0.080	10.9×	1.11	11.1×	0.078	0.56
	64^3	0.082	8.50×	4.78	8.25×	0.081	0.51*
	128^3	0.083	5.79×	4.78	8.47×	0.082	0.50*
	256^3	0.083	7.60×	4.78	11.2×	0.082	0.61*

C.4 Timing spread

Tables C.6 and C.7 give the spread behind the medians of Tables 3, C.5 and 1. Spread pools test cases and repetitions, so for iterative solvers it includes case-to-case variation in iteration count. Heat records store only the 10th and 90th percentiles. Burgers 2D full-order solves were timed once per case. Small meshes have heavy upper tails: at Burgers 2D 256^2 the fastest full-order query (14.18 ms) is faster than the median fast query (14.24 ms).

Table C.6: Timing spread for Tables 3 and C.5: median [minimum, maximum] in ms (heat: [10th, 90th percentile]). *Matched FOM* is the fast setting's own matched rung.

Problem	Mesh	Accurate	Fast	Matched FOM (fast)
Poisson 2D	256^2	0.56 [0.36, 2.52]	0.38 [0.21, 0.84]	10.3 [8.81, 11.6]
	1024^2	3.10 [2.86, 3.70]	1.18 [0.92, 1.77]	83.4 [74.3, 93.1]
	2048^2	11.7 [11.1, 12.3]	3.41 [3.08, 3.99]	594 [548, 678]
	4096^2	41.9 [41.2, 43.0]	12.1 [11.4, 12.9]	3866 [3584, 4421]
L-shape	256^2	0.32 [0.28, 1.05]	0.30 [0.26, 1.30]	8.88 [7.27, 9.80]
	512^2	0.52 [0.51, 0.64]	0.45 [0.44, 0.55]	21.8 [18.9, 24.3]
	1024^2	2.05 [1.83, 2.47]	1.73 [1.52, 1.93]	85.2 [74.7, 94.0]
	2048^2	3.81 [3.66, 3.99]	3.06 [2.91, 3.29]	298 [269, 326]
Poisson 3D	32^3	0.32 [0.25, 2.36]	0.27 [0.21, 1.70]	1.20 [1.03, 2.53]
	64^3	0.44 [0.40, 1.29]	0.30 [0.25, 1.01]	2.49 [1.89, 3.13]
	128^3	0.89 [0.88, 1.06]	0.42 [0.42, 0.64]	8.43 [7.19, 8.77]
	256^3	6.16 [5.98, 7.70]	2.50 [2.37, 3.18]	99.9 [91.4, 105]
Heat 2D	1024^2	0.64 [0.62, 1.37]	0.34 [0.32, 0.35]	27.9 [26.9, 29.8]
	2048^2	1.96 [1.95, 2.81]	0.83 [0.82, 0.86]	176 [166, 183]
	4096^2	8.42 [8.02, 8.74]	3.97 [3.59, 4.16]	1022 [974, 1087]
Heat 3D	32^3	0.31 [0.29, 1.27]	0.22 [0.20, 0.25]	2.48 [2.25, 2.60]
	64^3	0.64 [0.62, 1.52]	0.30 [0.29, 0.31]	2.48 (not extracted)
	128^3	2.99 [2.97, 3.44]	0.83 [0.81, 0.85]	7.03 (not extracted)
	256^3	23.6 [23.2, 23.9]	6.41 [5.98, 6.67]	71.9 (not extracted)
Burgers 2D	256^2	45.2 [33.2, 51.6]	14.2 [11.6, 17.8]	8.28 [7.58, 13.2]
	512^2	46.7 [34.4, 53.7]	15.1 [12.3, 18.7]	13.3 [12.2, 21.9]
	1024^2	49.1 [47.0, 56.9]	19.1 [16.8, 23.9]	32.6 [29.9, 56.3]
	2048^2	90.4 [87.4, 102]	35.2 [34.3, 37.3]	156 [116, 306]
	4096^2	119 [115, 131]	46.6 [45.4, 49.8]	647 [480, 1331]
Burgers 3D	64^3	81.1 [79.7, 93.5]	8.28 [8.00, 14.6]	7.72 [7.58, 8.23]
	128^3	85.0 [83.6, 96.6]	9.54 [9.44, 15.4]	27.4 [27.3, 28.2]
	256^3	90.7 [87.6, 103]	35.5 [34.5, 46.2]	199 [198, 217]
Navier–Stokes	32^3	5.76 [5.63, 6.35]	1.50 [1.48, 1.85]	5.33 [5.20, 5.73]
(original)	64^3	8.13 [8.03, 9.16]	2.78 [2.74, 3.62]	23.7 [23.3, 24.3]
	96^3	15.8 [15.5, 17.2]	8.83 [8.70, 10.1]	127 [126, 136]

Table C.7: Timing spread for Table 1 (seed 0, 72 timed queries per cell: eight cases $\times$ nine repetitions): median [minimum, maximum] in ms, and the matched solver chosen for that cell.

Setting	64^3	96^3	128^3
Width 64	16.1 [15.0, 17.4]; RK4 88^3, 112 ms	17.5 [17.1, 18.5]; RK4 88^3, 112 ms	30.8 [29.7, 32.4]; RK4 88^3, 112 ms
Width 96	17.1 [16.0, 18.6]; CNAB2 100^3, 201 ms	26.3 [25.7, 27.0]; CNAB2 108^3, 251 ms	43.2 [42.7, 44.1]; CNAB2 108^3, 299 ms
Width 128	30.7 [29.6, 31.8]; CNAB2 100^3, 201 ms	47.8 [46.7, 49.4]; CNAB2 112^3, 333 ms	62.4 [60.9, 64.0]; CNAB2 112^3, 334 ms
Width 128, fast	19.2 [18.3, 20.1]; CNAB2 100^3, 201 ms	30.6 [29.3, 31.7]; CNAB2 112^3, 333 ms	51.0 [49.7, 52.0]; RK4 108^3, 309 ms
Width 192	52.0 [50.8, 53.4]; RK4 108^3, 308 ms	67.2 [66.0, 68.4]; RK4 120^3, 414 ms	121 [120, 122]; RK4 120^3, 416 ms
Width 192, fast	24.6 [23.5, 25.6]; CNAB2 108^3, 250 ms	39.8 [38.9, 40.8]; CNAB2 120^3, 398 ms	67.6 [66.2, 68.8]; RK4 112^3, 346 ms

Seeds. The models of Table 1 were trained three times with independent training trajectories; median errors differ by less than 0.03 percentage points between seeds at widths 96 and above. No other model was retrained for this table. Three Burgers 2D retrains give 0.93/0.99/0.95 % on development cases at one dense-ladder setting. In a seed audit with three retrains per setting, two deployed Burgers 2D settings did not requalify their quadrature check on every retrain ($R' = 128$ failed on all three retrains, $R' = 384$ on two of three); they are marked § in Table D.1.

Full-order and solver details. The Burgers FOM is Newton's method with BiCGStab inner solves (at most 200 iterations, Jacobian–vector products by forward-mode differentiation) preconditioned by the exact inverse of the implicit Helmholtz operator $(I + \Delta t\, \nu\, \lambda)^{-1}$ (advection dropped); Newton stops when the residual norm falls below the nonlinear tolerance times the previous state's norm, or after 20 iterations, and always takes the full step (no line search or damping). Each named setting fixes Δt, the nonlinear and the linear tolerance.

Table C.8: Problem specification: meshes, time stepping and cohorts.

PDE	equation, domain, boundary	meshes	time stepping	cohorts
Burgers 2D	$u_t + u(u_x + u_y) = \nu\Delta u$, $(0,1)^2$, $u\|_{\partial\Omega} = 0$	256^2–4096^2	$\Delta t = 0.005$, backward Euler, sign-upwind	6 development cases; 32 validation; 64 test at 256^2–4096^2
Poisson 2D	$-\Delta u = f$, $(0,1)^2$, $u\|_{\partial\Omega} = 0$	256^2–4096^2	none (elliptic)	12 development / 32 test sources
Heat 2D, wide bank	$u_t = \kappa\Delta u$, $(0,1)^2$	1024^2–4096^2	Crank–Nicolson, $\Delta t = 0.025$	16 validation cases (selection); 16 test cases
Poisson, L-shape	$-\Delta u = f$, $(0,1)^2 \setminus [\frac{1}{2},1)^2$	256^2–2048^2	none	3072 / 256 / 32 / 32 sources (train / selection / development / test)
Navier–Stokes 3D, harder	as Table C.9, $\beta = 8$, $\nu/4$; periodic cube	64^3–128^3	implicit midpoint, 10–40 steps to $T = 0.2$	448 / 64 head trajectories per model; 64 development; 128 fresh test (32-case earlier cohort for Table E.1)
Cylinder (DFG 2D-2)	incompressible Navier–Stokes, Re 90–110	P2/P1, 96,781 unknowns	BDF2 Galerkin, step 1/400–1/3200	train Re 90/100/110; development Re 95; held-out Re 105; out of band Re 80/120

Table C.9: Sampled problem families.

PDE	equation and boundary	sampled family
Burgers 2D	$u_t + u(u_x + u_y) = \nu\Delta u$ on $(0,1)^2$, $u = 0$ on $\partial\Omega$	$u_0 = a\exp(-\|x-c\|^2/2w^2)$, $c_i \sim U(0.15, 0.85)$, $w \sim U(0.05, 0.20)$, $a \sim U(0.5, 2)$; $\nu \sim \log U(0.01, 0.1)$; outputs $t \in \{0.05, \ldots, 0.25\}$
Burgers 3D	$u_t + u(u_x + u_y + u_z) = \nu\Delta u$ on $(0,1)^3$, $u = 0$ on $\partial\Omega$	$u_0 \propto a\,\mu(x)\sum_{b\le B}\rho_b \exp(-\|x-c_b\|^2/2w_b^2)$, $\mu = 64\prod_i x_i(1-x_i)$, $B \in \{1,2,3\}$, $c_{b,i} \sim U(0.2, 0.8)$, $w_b \sim U(0.10, 0.20)$, $\rho_b \sim U(0.5, 1)$, peak normalised to $a \sim U(0.5, 2)$; $\nu \sim \log U(0.01, 0.1)$; backward Euler, sign-upwind; outputs $t \in \{0.05, \ldots, 0.25\}$
Poisson 2D	$-\Delta u = f$ on $(0,1)^2$, $u = 0$ on $\partial\Omega$	$f = a\exp(-\|x-c\|^2/2w^2)$, $c_i \sim U(0.15, 0.85)$, $w \sim \log U(0.02, 0.1)$, $a \sim U(0.5, 2)$
Poisson, L-shape	same source family on $(0,1)^2 \setminus [\frac{1}{2},1)^2$	as above; sources centred in the removed quadrant rejected
Heat 2D	$u_t = \kappa\Delta u$ on $(0,1)^2$, $u = 0$ on $\partial\Omega$, $\kappa = 0.02$ fixed	$u_0 = a\,16x_1(1-x_1)x_2(1-x_2)\exp(-\|x-c\|^2/2w^2)$, $c_i \sim U(0.35, 0.65)$, $w \sim U(0.10, 0.15)$, $a \sim U(0.8, 1.2)$; outputs $t \in \{0, 0.1, \ldots, 0.5\}$; Crank–Nicolson $\Delta t = 0.025$
Poisson 3D	$-\Delta u = f$ on $(0,1)^3$, $u = 0$ on $\partial\Omega$	$f = a\,64\prod_i x_i(1-x_i)\exp(-\|x-c\|^2/2w^2)$, $c_i \sim U(0.35, 0.65)$, $w \sim U(0.10, 0.15)$, $a \sim U(0.8, 1.2)$
Heat 3D	$u_t = \kappa\Delta u$ on $(0,1)^3$, $u = 0$ on $\partial\Omega$, $\kappa = 0.02$ fixed	$u_0 = a\prod_i 4x_i(1-x_i)\exp(-\|x-c\|^2/2w^2)$, $c_i \sim U(0.35, 0.65)$, $w \sim U(0.10, 0.15)$, $a \sim U(0.8, 1.2)$; outputs $t \in \{0, 0.1, \ldots, 0.5\}$; Crank–Nicolson $\Delta t = 0.025$
Navier–Stokes 3D	$u_t + (u\cdot\nabla)u = -\nabla p + \nu\Delta u$, $\nabla\cdot u = 0$ on the periodic cube $[0,1)^3$	u_0 the curl of two potentials $\exp\big(2\sum_i(\cos 2\pi(x_i - c_i) - 1)\big)$ along z and $(1,1,0)/\sqrt{2}$, amplitude 0.2; centre $c \sim U(0,1)^3$, separation $\sim U(0.12, 0.24)$ along $(1,1,1)/\sqrt{3}$, second strength $\sim U(0.6, 1.4)$; $\nu \sim \log U(0.002, 0.01)$; reference CNAB2 $\Delta t = 0.001$; outputs $t \in \{0, 0.04, \ldots, 0.2\}$

C.5 Headline settings, times and timing protocol

The settings behind Table 3, all from one frozen model per problem with no retraining: Poisson (square and cube) and heat use the bank span at two widths ($R' = 512/128$ square, $128/32$ cube, $128/32$ heat 2D, $320/128$ heat 3D at 32^3 and $320/64$ above, Crank–Nicolson stepping); Navier–Stokes uses the head $k = 8$ and the span $R' = 16$ (the cheapest span setting below 5 %); the L-shape uses the span $R' = 128/64$ (the widest span its tests support is 128); Burgers uses the span $R' = 384$ at every mesh and $R' = 96$ (256^2–1024^2) or $R' = 128$ (2048^2, 4096^2). Burgers 3D uses one coordinate bank ($R = 512$, trained once on 32^3, 64^3 and 128^3 fields) with the span $R' = 512$ and $R' = 192$ ($R' = 256$ at 256^3), a fixed-sweep Levenberg–Marquardt solve and a precomputed

quadratic tensor for the advection term (about $4R'$ tests); its settings were chosen by the same rule on 64 validation cases and its rows are 32 test cases, all on H200 GPUs. All Burgers 2D rows were timed on A100 80 GB GPUs. The three-dimensional Poisson rows use 64 test cases. The FOM setting can differ between meshes of a series.

Table C.10 gives the settings and measured times behind every row of Table 3. Every cost and error come from the same invocation. Timings use GPU warm-up and synchronisation, double precision and the highest matrix-multiplication precision; all repetitions, including outliers, enter the median, and no time is borrowed from another allocation. Requested full fields and initialisation are charged; host transfer is not.

The Poisson and three-dimensional heat CG comparators are unpreconditioned; heat CG is warm-started from the previous time level.

Table C.10: Supporting data for Table 3, including the full-order setting the rule selected for each row: the two settings of each frozen model, median GPU query times (ms) and status. "lattice" is the fixed 63×63 lattice of Appendix A.6 (the submission labelled it "EQ"); "tensor" is the precomputed quadratic advection tensor used in three dimensions. With the host transfer of the full field included, the accurate and fast Poisson 2D speedups are $7.05\times$ and $7.72\times$ at 256^2 and $46.3\times$ and $66.2\times$ at 4096^2, and the accurate Poisson 3D speedups are 2.78–$3.63\times$ at 64^3–256^3. Heat errors are measured against the exact time evolution on the same grid, so the NM-ROM and CN–CG errors both include the Crank–Nicolson time error.

Problem	Mesh	Accurate	Fast	Accurate ms	Fast ms	FOM setting	FOM ms	Status
Poisson 2D	256^2	span, R'=512	span, R'=128	0.56	0.38	CG, rtol 3×10^{-2}	10.3	test
Poisson 2D	1024^2	span, R'=512	span, R'=128	3.10	1.18	CG, rtol 3×10^{-2}	96.4	test
Poisson 2D	2048^2	span, R'=512	span, R'=128	11.66	3.40	CG, rtol 3×10^{-2}	751.0	test
Poisson 2D	4096^2	span, R'=512	span, R'=128	41.92	12.11	CG, rtol 10^{-1}	4508.8	test
Poisson, L-shape 2D	256^2	span, R'=128	span, R'=64	0.32	0.30	CG, rtol 10^{-2}	10.3	test
Poisson, L-shape 2D	512^2	span, R'=128	span, R'=64	0.52	0.45	CG, rtol 10^{-2}	26.2	test
Poisson, L-shape 2D	1024^2	span, R'=128	span, R'=64	2.05	1.73	CG, rtol 3×10^{-2}	85.2	test
Poisson, L-shape 2D	2048^2	span, R'=128	span, R'=64	3.81	3.06	CG, rtol 3×10^{-2}	298.5	test
Heat 2D	1024^2	span, R'=128	span, R'=32	0.64	0.34	CN–CG, Δt=0.025, rtol 10^{-4}	67.00	test
Heat 2D	2048^2	span, R'=128	span, R'=32	1.96	0.83	CN–CG, Δt=0.025, rtol 10^{-4}	424.37	test
Heat 2D	4096^2	span, R'=128	span, R'=32	8.42	3.97	CN–CG, Δt=0.025, rtol 10^{-3}	1945.97	test
Burgers 2D	256^2	span, R'=384, M=1536, lattice	span, R'=96, lattice	45.2	14.2	Newton–BiCGStab, tol 3×10^{-3}	17.4	test
Burgers 2D	512^2	span, R'=384, M=1536, lattice	span, R'=96, lattice	46.7	15.1	Newton–BiCGStab, tol 3×10^{-3}	29.1	test
Burgers 2D	1024^2	span, R'=384, M=1536, lattice	span, R'=96, lattice	49.1	19.1	Newton–BiCGStab, tol 3×10^{-3}	74.1	test
Burgers 2D	2048^2	span, R'=384, M=1536, lattice	span, R'=128, lattice	90.4	35.2	Newton–BiCGStab, tol 3×10^{-3}	306.8	test
Burgers 2D	4096^2	span, R'=384, M=1536, lattice	span, R'=128, lattice	119.4	46.6	Newton–BiCGStab, tol 3×10^{-3}	1282.5	test
Poisson 3D	32^3	span, R'=128	span, R'=32	0.32	0.27	CG, rtol 10^{-2}	1.41	test
Poisson 3D	64^3	span, R'=128	span, R'=32	0.44	0.30	CG, rtol 10^{-2}	3.57	test
Poisson 3D	128^3	span, R'=128	span, R'=32	0.89	0.42	CG, rtol 10^{-2}	12.1	test
Poisson 3D	256^3	span, R'=128	span, R'=32	6.16	2.50	CG, rtol 3×10^{-2}	118.8	test
Heat 3D	32^3	span, R'=320	span, R'=128	0.31	0.22	CN–CG, Δt=0.025, rtol 10^{-4}	3.38	test
Heat 3D	64^3	span, R'=320	span, R'=64	0.64	0.30	CN–CG, Δt=0.025, rtol 10^{-4}	5.42	test
Heat 3D	128^3	span, R'=320	span, R'=64	2.99	0.83	CN–CG, Δt=0.025, rtol 10^{-4}	17.33	test
Heat 3D	256^3	span, R'=320	span, R'=64	23.61	6.41	CN–CG, Δt=0.025, rtol 10^{-4}	179.42	test
Burgers 3D	64^3	span, R'=512, Δt=0.005, tensor	span, R'=192, Δt=0.01, tensor	81.11	8.28	Newton–BiCGStab, Δt=0.005, tol 10^{-3}	17.26	test
Burgers 3D	128^3	span, R'=512, Δt=0.005, tensor	span, R'=192, Δt=0.01, tensor	85.00	9.54	Newton–BiCGStab, Δt=0.01, tol 10^{-3}	48.68	test
Burgers 3D	256^3	span, R'=512, Δt=0.01, tensor	span, R'=256, Δt=0.01, tensor	90.70	35.50	Newton–BiCGStab, Δt=0.01, tol 10^{-2}	198.69	test
Navier–Stokes 3D	32^3	head, k=8	bank span, R'=16, Δt=0.04, 2 sweeps	5.76	1.50	CNAB2 (spectral), 40 steps	5.33	test
Navier–Stokes 3D	64^3	head, k=8	bank span, R'=16, Δt=0.04, 2 sweeps	8.13	2.78	CNAB2 (spectral), 50 steps	23.72	test
Navier–Stokes 3D	96^3	head, k=8	bank span, R'=16, Δt=0.02, 3 sweeps	15.82	8.83	CNAB2 (spectral), 70 steps	126.78	test

Training/evaluation separation (Burgers). The 4608 training trajectories of the Burgers checkpoint were regenerated from their seeds and compared with every evaluation cohort (the 32 validation cases of Table B.2, the 64 test cases and the 6 development cases): no cohort shares a parameter vector, or any single parameter value, with the training set.

D TUNABILITY LADDERS

Table D.1 gives the bank-width ladders behind §5.5.

E HARDER NAVIER–STOKES FAMILY AND CYLINDER FLOW

E.1 HARDER NAVIER–STOKES FAMILY

Family. The initial field is the curl of two potentials $\exp\left(\beta\sum_i(\cos 2\pi(x_i-c_i)-1)\right)$ as in Table C.9, with sharpness $\beta=8$ (original family: $\beta=2$), blob width scaled by $s=\sqrt{2/\beta}$, amplitude $0.2s$ and separation $(0.12, 0.24)\,s$, and viscosity $\nu\sim\log U(5\times10^{-4}, 2.5\times10^{-3})$, a quarter of the original range; horizon $T=0.2$, outputs every 0.04. It was chosen from five candidate families of increasing difficulty by two criteria fixed before any reduced model was run: the cheapest solver reaching 1 % on probe cases must need a fine mesh (here CNAB2 at 96^3, 167 ms; the original family needs RK4 at

Table C.11: Solver iterations per query and where the query time goes. On Poisson and heat the query is a direct solve; *input* is projecting the forcing or fitting the initial state, *solve* the reduced solve including all time steps, and *output* rebuilding the field at the output times. These stage times come from separately compiled stages in the same jobs, so they do not add exactly to the query time. Burgers: Levenberg–Marquardt iterations per query, median over the cases (largest number in one step in brackets): in two dimensions over 50 time steps of the six development cases, every step stopped on the convergence test; in three dimensions over 50 ($\Delta t = 0.005$) or 25 ($\Delta t = 0.01$) steps of the 32 test cases. Navier–Stokes: 10 time steps of three Gauss–Newton sweeps each, plus 12 Gauss–Newton sweeps that fit the head's starting code to the projected initial field, starting from a stored training code. The Poisson 2D, Burgers 2D and Navier–Stokes rows are development runs, so their query times differ from Table C.10; the Burgers 2D 4096^2 row ran on an H200 GPU and the others on A100 GPUs, which is why it is faster than 2048^2. Times in ms.

Problem	Mesh	Setting	Iterations	Input	Solve	Output	Query
Poisson 2D	4096^2	span R'=512	0	0.99	<0.01	41.79	42.39
		span R'=128	0	0.74	<0.01	12.52	12.54
Heat 2D	4096^2	span R'=128	0	1.34	0.07	6.74	8.42
		span R'=48	0	1.10	0.07	3.09	4.74
Heat 3D	256^3	span R'=320	0	11.88	0.08	12.28	23.61
		span R'=128	0	5.81	0.08	5.42	10.68
Burgers 2D	256^2	span R'=384	52 (4)				45.71
		span R'=128	38.5 (4)				17.44
	512^2	span R'=384	52 (4)				45.91
		span R'=128	38.5 (4)				17.39
	1024^2	span R'=384	54 (5)				49.52
		span R'=128	51.5 (5)				21.31
	2048^2	span R'=384	54 (5)				90.09
		span R'=128	51.5 (5)				35.15
	4096^2	span R'=384	54 (5)				59.60
		span R'=128	51.5 (5)				24.06
Burgers 3D	64^3	span R'=512	51 (3)				81.11
		span R'=192	27 (5)				8.28
	128^3	span R'=512	51 (3)				85.00
		span R'=192	26 (5)				9.54
	256^3	span R'=512	28 (5)				90.70
		span R'=256	27 (5)				35.50
Navier–Stokes 3D	32^3	head k=8	30 + 12				6.26
		span R'=16	30				3.65
	64^3	head k=8	30 + 12				8.19
		span R'=16	30				4.64
	96^3	head k=8	30 + 12				15.96
		span R'=16	30				9.16

Table C.12: Supporting data for Table B.2: its 256^2 and 512^2 rows. Our rows use the unoptimised dense reference path (no empirical quadrature, stopping tolerance 10^{-6}), so no times are compared. The shallow masked-autoencoder NM-LSPG of Y. Kim et al. (2022) reproduced its published result on their own 2D Burgers benchmark (their Sec. 6.2) on the third of three planned attempts (sigmoid activation; median $1.45\,\%$ against a $1.5\,\%$ bar; published $<1\,\%$). Its published encoder width ($2n$) needs 136 GB to train at 256^2 and 2183 GB at 512^2, beyond the 77 and 135 GB devices; at 512^2 its encoder, capped at width 4096, trained only 229–231 epochs in its 4800 s budget, so those errors partly reflect a training-time limit. The data-matched row is trained on our bank's 576 trajectories.

Mesh	Method	k	Worst (%)	Median (%)	MB
256^2	Kim et al. NM-LSPG	8	163.93	43.34	2480
256^2	Kim et al. NM-LSPG	16	145.40	43.87	2567
256^2	Kim et al. NM-LSPG	32	127.51	43.61	2742
256^2	Kim et al. NM-LSPG, data-matched	16	120.33	38.71	2567
256^2	POD-LSPG	8	55.52	22.72	18
256^2	POD-LSPG	16	41.53	10.45	30
256^2	POD-LSPG	32	24.78	5.65	55
256^2	This work, head ($k = 16$)	16	6.79	0.66	625
512^2	Kim et al. NM-LSPG	8	175.71	55.48	9959
512^2	Kim et al. NM-LSPG	16	174.99	58.56	10511
512^2	Kim et al. NM-LSPG	32	252.33	56.72	10879
512^2	POD-LSPG	8	55.82	24.08	71
512^2	POD-LSPG	16	42.01	13.83	122
512^2	POD-LSPG	32	25.63	7.74	222
512^2	This work, head ($k = 16$)	16	7.72	0.69	2368

Table C.13: Three-dimensional configurations, read from the run records. Domains are the unit cube with homogeneous Dirichlet data, except Navier–Stokes, which is periodic.

Problem	Mesh	Cases	Reduced model	Named FOM	Residual
Poisson 3D	32^3–256^3	64 test	span $R' \in \{16, \ldots, 128\}$	CG, rtol 10^{-2}, no preconditioner	exact (linear)
Heat 3D	32^3–256^3	64 test (all times)	span $R' \in \{320, \ldots, 32\}$ ($R = 320$)	CN–CG, $\Delta t = 0.025$, rtol 10^{-4} (the rule's choice at every mesh)	exact (linear)
Burgers 3D	64^3, 128^3, 256^3	64 validation (settings); 32 test	coordinate bank $R = 512$; span $R' \in \{512, \ldots, 64\}$, $\Delta t \in \{0.005, 0.01\}$	Newton–BiCGStab, setting per row by the rule	precomputed quadratic tensor, checked on reached states
Navier–Stokes 3D	32^3, 64^3, 96^3 periodic	16 development / 32 test cases	POD bank $R = 64$ built per mesh, in a moving frame; head $k = 8$; span $R' \in \{64, 48, 32, 16, 8\}$	CNAB2 (spectral), 40–70 steps (per row by the rule)	precomputed quadratic tensor

Table D.1: Bank-width tunability at the finest mesh. Each block is one frozen model, and within a cohort every speedup divides the median time of *one* full-order setting (the *FOM* row, the fastest at least as accurate as the block's most accurate setting, in the same allocation) by the setting's time (ms). Error is the worst same-grid relative L^2 error (%), over evolved times for Navier–Stokes and Burgers and over all times for heat. Test cases were never used for selection and share the development settings; for Burgers 3D the development column is the 64 validation cases on which its settings were chosen, and only the two selected widths were run on the test cases. The Burgers 2D test cases ran on an A100 GPU and the development cases on an H200 GPU. Heat is shown at 2048^2. The Burgers 2D test column of $R' = 128$ and $R' = 64$ was run after the fast width was chosen on test errors (Table 3). [§] Setting that did not requalify its quadrature check on every one of three retrains (Appendix C.4).

	Development cohort			Test cohort		
Setting	Err. (%)	ms	Speedup	Err. (%)	ms	Speedup
Poisson 2D, 4096^2 *(12 development / 32 test cases)*						
Head, k=32, R'=512	3.15	43.4	103×	4.48	43.0	105×
Bank span, R'=512	0.74	42.4	106×	0.83	41.9	108×
Bank span, R'=384	0.77	32.5	138×	1.23	32.0	141×
Bank span, R'=256	0.94	22.6	199×	2.05	22.0	205×
Bank span, R'=128	2.31	12.5	358×	4.06	12.1	372×
Bank span, R'=64	5.33	7.1	631×	6.61	6.7	672×
Bank span, R'=32	9.17	4.4	1022×	15.5	4.0	1136×
FOM: CG, rtol 10^{-1}	0.45	4490		0.58	4509	
Navier–Stokes 3D, moving frame, 96^3 *(16 development / 32 test cases)*						
Head, k=8	0.15	16.0	6.67×	0.21	15.8	7.79×
Bank span, R'=64	0.62	15.4	6.92×	1.12	15.3	8.05×
Bank span, R'=48	1.00	12.9	8.27×	1.26	12.6	9.76×
Bank span, R'=32	1.26	11.6	9.14×	1.46	11.5	10.7×
Bank span, R'=16	2.96	9.2	11.6×	3.24	9.2	13.4×
Bank span, R'=8	5.55	8.2	13.0×	6.91	8.1	15.2×
FOM: CNAB2 (spectral), 60 / 70 steps	0.063	106		0.056	123	
Burgers 2D, 4096^2 *(6 development / 64 test cases)*						
Head, k=16, R'=512	2.41	41.5	12.6×	—	—	—
Bank span, R'=512 (quadrature rule failed its check)	0.30	77.6	6.74×	—	—	—
Bank span, R'=384[§]	0.22	59.6	8.78×	1.35	119	10.7×
Bank span, R'=256	0.74	42.2	12.4×	—	—	—
Bank span, R'=128[§]	1.89	24.1	21.7×	3.11	46.6	27.5×
Bank span, R'=64	4.83	16.1	32.6×	8.24	28.8	44.5×
Bank span, R'=32	10.7	12.1	43.2×	—	—	—
FOM: Newton–BiCGStab, tol. 3×10^{-3}	0.050	523		0.14	1282.5	
Burgers 3D, Δt=0.01, 256^3 *(64 validation / 32 test cases)*						
Bank span, R'=512	2.43	89.7	2.22×	2.20	90.7	2.19×
Bank span, R'=384	3.10	58.4	3.41×	—	—	—
Bank span, R'=256	3.84	34.0	5.86×	3.77	35.5	5.60×
Bank span, R'=192	5.65	25.7	7.74×	—	—	—
Bank span, R'=128	10.5	18.0	11.1×	—	—	—
Bank span, R'=96	14.4	13.9	14.3×	—	—	—
Bank span, R'=64	21.9	9.5	20.9×	—	—	—
FOM: Newton–BiCGStab, Δt=0.01, tol. 10^{-2}	1.27	199		1.52	199	
Heat 2D, Crank–Nicolson, Δt=0.025, 2048^2 *(16 test cases)*						
Head, k=8, R'=128	—	—	—	1.36	31.5	13.5×
Bank span, R'=128	—	—	—	0.13	1.96	216×
Bank span, R'=96	—	—	—	0.15	1.63	261×
Bank span, R'=64	—	—	—	0.19	1.28	332×
Bank span, R'=48	—	—	—	0.28	1.05	405×
Bank span, R'=32	—	—	—	1.08	0.83	512×
Bank span, R'=16	—	—	—	8.68	0.62	687×
FOM: CN–CG, $\Delta t = 0.025$, rtol 10^{-4}	—	—	—	0.045	424	

32^3, 3.8 ms), and the held-out POD projection floor at rank 64 must stay near 1 %. The reference is CNAB2 at 192^3 with $\Delta t = 2.5\times10^{-4}$, stored truncated to 128^3.

Model. Per mesh (64^3, 96^3, 128^3) a POD bank of rank $R \in \{64, 96, 128, 192\}$ is built from centred training snapshots and evaluated in the moving frame of Appendix A.5; the head ($k = 8$, width 256, linear skip) is trained per mesh as an auto-decoder on 512 training trajectories (448 training, 64 validation). Number of tests $M \in \{292, 584, 1168, 2336\}$, step count $\{10, 20, 40\}$ and three Gauss–Newton sweeps per step were selected on 64 development cases by a rule fixed in advance (smallest error, then cheapest within 5 %); the *fast* rows take the cheapest setting within the same rule's tolerance. Each of the three models re-draws its training trajectories, banks and heads; the fresh 128-case cohort was generated after the settings were frozen.

Solver ladder and timing. The comparator is a GPU Fourier pseudo-spectral solver (CNAB2 or RK4) on meshes 32^3–128^3 with step counts from 5 to 640, 450 stable configurations in all, scored on the same 128 cases. For each setting the matched solver is the fastest configuration whose worst error on the cohort is at or below the setting's (any mesh); times are solve-only (initial field already on the solver's mesh, output left on it). Setting and solver are timed interleaved in one block on eight cases with nine repetitions, with sentinel arms re-timed alone before and after the block; a cell is admitted only if every sentinel ratio stays below 1.25. Seed 1's timing block failed this gate twice at 64^3 and 96^3 (one 2.3 ms solver sentinel); each failed timing run was repeated once under the protocol's rule, and seed 1's errors are reported.

One coordinate network. Table E.1 gives the earlier 32-case test cohort on which one coordinate-network bank (a divergence-free potential network with a Fourier first layer, fitted once by variable projection to centred training snapshots) replaces the per-mesh POD banks. On the original family the same construction gives 0.18–0.20 % against 0.15 % for per-mesh POD.

Table E.1: Harder Navier–Stokes family, earlier 32-case test cohort: coordinate network (fitted once, rank 64 or 96) with the head, against per-mesh POD with the head re-timed in the same block, and with the 64^3 head reused unchanged on the finer meshes. Worst error (%), time and speedup over the matched any-mesh solver. Settings frozen on development cases; the POD and interpolated-POD rows had been evaluated on this cohort once before. Every rank-64 variant missed the pre-set mesh-transfer tolerance on development cases (projection error 6–8 % higher at 128^3 than at 64^3, limit 5 %), so a fallback rule chose it.

Mesh	Coordinate bank + head	Per-mesh POD + head	Rank-96 coordinate bank + head	64^3 head reused (coord. / interp. POD)
64^3	0.90 %, 12.7 ms, 14.2×	0.84 %, 12.1×	0.72 %, 13.0×	—
96^3	1.23 %, 16.7 ms, 6.7×	1.23 %, 6.7×	0.93 %, 11.4×	1.05 %, 8.8× / 1.10 %, 7.7×
128^3	1.39 %, 29.3 ms, 3.8×	1.17 %, 6.1×	0.91 %, 6.4×	1.06 %, 5.4× / 1.10 %, 4.9×

Batched throughput. Both our model and the matched spectral solver were batched with `jax.vmap` on the same GPU (the solver also in a faster real-FFT version); every batched output matches the batch-one code to 2×10^{-13} and the fresh-cohort errors are reproduced. Solver time over our time per query for the fast width-192 head (seed 0, end-to-end for our model):

Mesh	$B = 1$	$B = 8$	$B = 32$	$B = 128$
64^3	9.4×	39.8×	64.5×	75.1×
96^3	11.6×	35.7×	46.7×	48.4×
128^3	6.5×	15.4×	17.3×	17.1×

The solver already saturates the GPU at one query; our reduced solve batches well, and at 128^3 about 80 % of our batched time is output reconstruction. The 96^3, $B = 128$ cell was timed on an H200, all others on an A100.

Solve time against the mesh. At identical settings, reduced-solve time varies by 1–4 % from 64^3 to 128^3 within one A100 allocation. Extensions to 256^3 use resampled 128^3 banks with reduced operators identical to 10^{-15}; their timings come from separate allocations. Input projection and output reconstruction grow with the mesh, so the full query is not flat: at 256^3 it has no consistent speed advantage over the matched solver running on its own coarser mesh (0.56–1.27×). The cross-mesh timing study spans multiple allocations: matched-solver ratios are always formed within one

allocation, and cross-mesh wall-clock differences are descriptive, not paired hardware-controlled speedups.

E.2 Unsteady flow past a cylinder

Problem. DFG 2D-2 channel and cylinder (Schäfer and Turek, 1996), mean inflow 1, kinematic viscosity $0.1/\mathrm{Re}$, gradient-form outlet. Trajectories start from the steady state at Reynolds 20 with a ramped inflow and a one-off asymmetric kick; each records a window of 1.7 s (about five shedding periods), saved every 1/400 s. Training Reynolds numbers 90, 100, 110; development 95; held-out 105, opened once. The training data come from a curved P4/P3 Taylor–Hood reference with BDF2; the model lives on the P2/P1 export mesh with 96,781 unknowns.

Model. The bank represents the velocity minus a lift (the mean training velocity), which carries the inflow. Its boundary factor is a product of smooth distance factors to the inlet, the channel walls and the cylinder surface. The bank is trained on the transient training states and ordered as in §3.1; widths 16 and 24 are prefixes of the width-32 bank. Each velocity width r is paired with r pressure functions and one supremizer per pressure function, and the rollout is a fully implicit BDF2 Galerkin projection with all reduced operators assembled once. A POD basis of equal width, rolled out the same way, performs the same within 1 %.

Targets. Velocity RMS error $\leq 10^{-2}$ and maximum $\leq 3\times10^{-2}$, pressure RMS error $\leq 2\times10^{-2}$, relative Strouhal error $\leq 10^{-2}$, relative maximum-drag error $\leq 2\times10^{-2}$ and absolute maximum-lift error $\leq 2\times10^{-2}$, all fixed in advance.

Comparator. 62 finite-element configurations (mesh, element order P2/P1 or P3/P2, semi-implicit, Oseen or frozen-Oseen BDF2, step and refresh interval) were screened on the development case; the nine cheapest passing candidates were re-checked at Reynolds 105 and the passing ones timed with five repetitions. The cheapest is P3/P2 on a coarser mesh (16,912 unknowns), step 1/300, refresh every 30 steps: 42.2 s (37.5–49.4 s), velocity error 9.4×10^{-3}. Model and finite element run on the same 16-thread CPU node, the model timed before and after the finite-element repetitions.

Table E.2: Cylinder, step 1/800: velocity RMS error and targets met (of six). Long window: three times the standard window at Reynolds 105. Out of band: Reynolds 80 and 120. Operators: the best of six trained for this comparison (Table F.2).

	Long window, Re 105	Re 80	Re 120
Width 32	2.4×10^{-3}, 6/6	1.2×10^{-2}, 5/6	1.3×10^{-2}, 4/6
Width 24	4.0×10^{-3}, 6/6	5.1×10^{-2}, 1/6	2.5×10^{-2}, 3/6
Width 16	1.9×10^{-2}, 3/6	7.2×10^{-2}, 1/6	3.0×10^{-2}, 3/6
Best operator	3/6	3/6	4/6

Multigrid comparator. We replaced the sparse LU of the cheapest passing finite-element run by algebraic multigrid (pyamg smoothed aggregation): a block-triangular preconditioner with one V-cycle per velocity component and a Cahouet–Chabard pressure Schur-complement approximation inside flexible GMRES, with the hierarchies rebuilt exactly where the LU was refactored. At Reynolds number 105 every variant met all six targets with values identical to LU. Against LU timed in the same job (five repeats, 16 threads, same node), the AMG-preconditioned solver was $2.38\times$ slower at the tolerance selected on the development case and $1.87\times$ slower at the loosest passing tolerance; AMG inner solves were $21\times$ slower. At this size the sparse direct solver is the faster comparator. Compiled multithreaded multigrid (hypre, PETSc) was not tested.

F Baselines added in the revision

F.1 Neural-field ROMs

The CROM-style model is a neural field of the coordinates conditioned on 16 (or 32) latent coordinates, evolved online by solving the discretised PDE for the latent velocity at every step, which requires differentiating the spatial network at the quadrature nodes. The CoLoRA-style model is a hyper-network that maps the five parameters of the initial condition and time to low-rank adaptations of a

neural field; it solves no equation online. Both are our re-implementations, trained on our Burgers 2D training data, with every choice made on validation data. A CoLoRA-style decoder driven by our online residual solve gave 31.4 % worst-case error and is not used. Disclosure: the first test job was cancelled for its time limit after printing two completed CROM-style rows, which we read before resubmitting; the selection file was unchanged and the rerun reproduced those rows.

F.2 Neural operators with matched data

Table F.1 gives every trained operator of Table 4. FNO and U-Net were trained on exactly the 4,608 trajectories our head uses, for up to 8.1 hours each (a 4.6-hour run plus one 3.5-hour continuation), with the architecture selected on validation data. Most runs reached the time budget before our convergence rule, so these results hold under the stated budgets, not at convergence.

Table F.1: Burgers 2D, 64 test cases. Left: 256^2 (training resolution). Right: 1024^2 without retraining (operators applied zero-shot). F0 is the submission's FNO architecture; F2 the variant selected on validation.

		256^2		1024^2
Method	Trajectories	Worst (%)	Median (%)	Worst (%)
Ours, span $R' = 384$	576 + 4,608	0.875	0.026	1.25
Ours, span $R' = 96$	576 + 4,608	3.85	0.53	4.25
FNO F0, seeds 0/1/2	4,608	0.48/0.44/0.49	0.22/0.20/0.23	3.17/2.99/3.04
FNO F2 (selected)	4,608	0.48	0.19	2.98
U-Net, seeds 0/1/2	4,608	0.45/0.50/0.51	0.21/0.22/0.20	53.5/54.0/53.4
FNO F0	576	4.03	0.40	
U-Net	576	0.74	0.27	
FNO F0 / U-Net, rerun	128	9.78 / 5.01	2.61 / 1.23	

Harder Navier–Stokes. An FNO (81 M parameters, the better on validation of two sizes) trained on the head's 448/64 trajectory split gives 2.41/2.51/2.52 % worst-case error on the 32-case cohort of Table E.1; the coordinate-bank head gives 0.90/1.23/1.39 % on the same cohort. This is an accuracy-only comparison.

Cylinder. Table F.2 gives the six operators trained on the same three training trajectories (60 and 180 minutes of final training each). We report prediction errors and acceptance targets only.

Table F.2: Cylinder operators: velocity RMS error and targets met (of six) on the standard window, and targets met on the long window and out of band. The operator comparison is accuracy-only.

Operator, final training	Dev. Re 95	Held-out Re 105	Long window	Re 80	Re 120
Transolver-style, 60 min	8.3×10^{-3}, 4/6	6.9×10^{-3}, 4/6	2/6	2/6	3/6
Transolver-style, 180 min	3.7×10^{-3}, 4/6	5.3×10^{-3}, 4/6	3/6	3/6	4/6
FNO, 60 min	1.1×10^{-2}, 4/6	9.8×10^{-3}, 4/6	0/6	1/6	1/6
FNO, 180 min	9.0×10^{-3}, 6/6	9.9×10^{-3}, 6/6	0/6	0/6	1/6
DeepONet, 60 min	2.5×10^{-2}, 3/6	7.5×10^{-3}, 5/6	0/6	1/6	2/6
DeepONet, 180 min	3.5×10^{-2}, 2/6	1.2×10^{-2}, 4/6	1/6	0/6	1/6

F.3 Ablations and POD controls

Table F.3 collects the component ablations. Two POD controls bound what the coordinate network adds. (i) On periodic boxes a POD basis built on one mesh and interpolated to the others, used with the same head, transfers as well as the coordinate network (harder Navier–Stokes: 1.10 % at 96^3 and 128^3 with the 64^3 head, Table E.1); there the network provides a single mesh-free object, not accuracy. (ii) In an earlier in-sample study on Burgers 2D at 1024^2, a tuned POD basis with ECSW hyper-reduction (Farhat et al., 2014) matched or beat all eight bank settings tested, for example 1.35 % worst-case error in 0.14 of the time of our width-512 span at 2.17 %.

G Figures

The figures below were drawn from the stored run records of the experiments they name; each caption states its cohort.

Table F.3: Component ablations on Burgers 2D. Closures act on the same bank with 16 coordinates; POD-LSPG and the quadratic manifold use their own basis with 16 unknowns. Nonlinear term: the same span setting with the fixed lattice and with the dense residual (six development cases).

Comparison	Setting	Worst (%)	ms
Latent map, 16 coordinates, 256^2	nonlinear head	2.56	
	linear closure	56.9	
	quadratic closure	31.8	
	POD-LSPG, 16 modes	61.7	
Latent map, 16 unknowns, 1024^2 (64 test cases)	nonlinear head	8.42	35.3
	POD-LSPG	41.6	186.5
	quadratic manifold	34.1	758
Nonlinear term, span $R' = 384$, 1024^2	fixed 63×63 lattice	0.2108	49.3
	dense residual	0.2108	1,263
Head or span (development)	span $R' = 128$	1.10	$0.59\times$ head

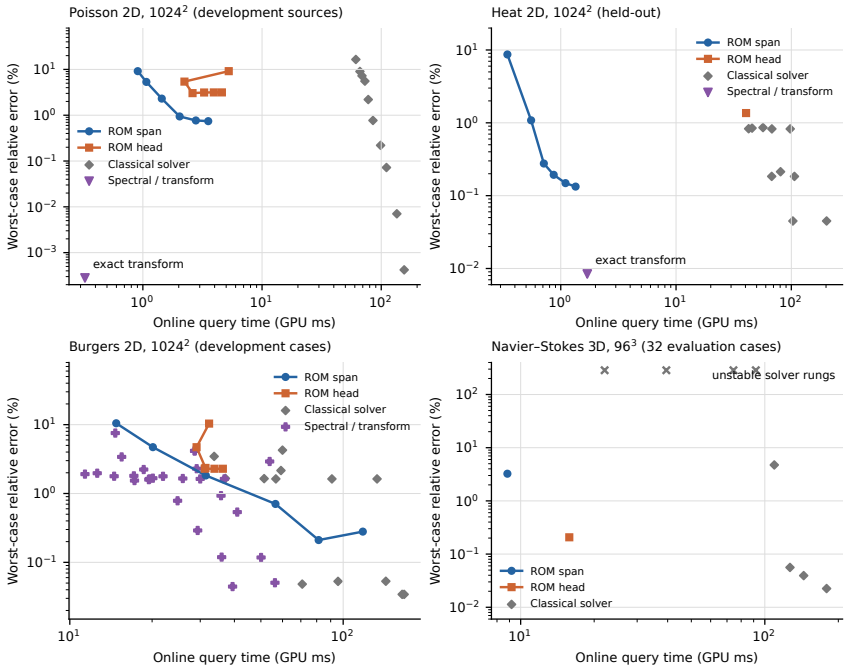


Figure G.1: Error against cost. Worst-case relative error against median online GPU time per query on four problems of the original benchmark set. Blue: span along its width ladder R'; orange: head; grey diamonds: the named full-order solver along its own ladder; violet: transform or pseudo-spectral solvers. Neural operators are compared by error in Tables 4 and F.1 and are omitted from this cost plot. Poisson 2D and Burgers 2D panels are development cohorts at 1024^2 (held-out errors are higher), heat 2D at 1024^2, original Navier–Stokes at 96^3. Points come from more than one job (5–20 % cross-job timing differences). On these problems the transform and pseudo-spectral solvers are faster at equal error, which the figure shows.

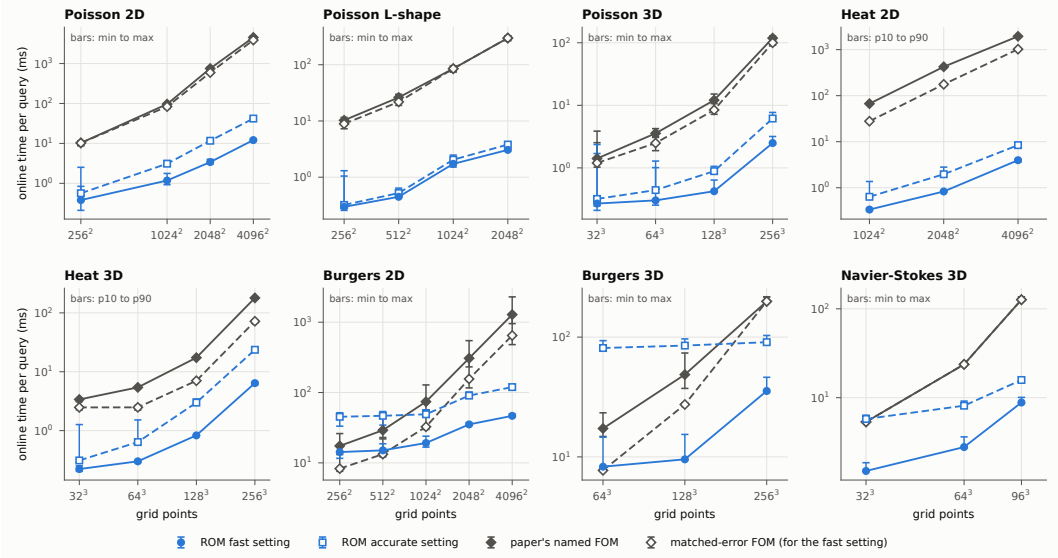


Figure G.2: Cost against mesh. Median online time per query against the number of grid points for the problems of Tables 3 and C.5. Blue solid: fast setting; blue dashed: accurate; grey solid: the submission's named solver; grey dashed: the fast setting's matched solver. Bars: minimum to maximum over stored repeats (heat: 10th to 90th percentile).

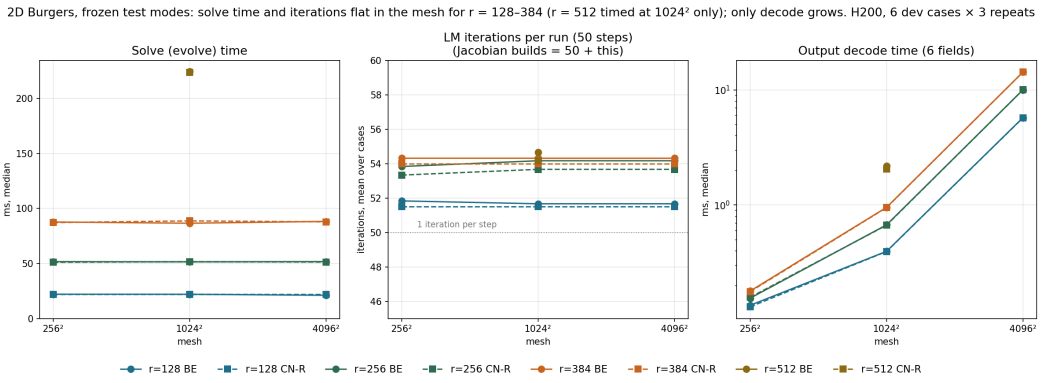


Figure G.3: The reduced evolution does not grow with the mesh. Burgers 2D, the submission's frozen bank with a later off-mesh solver variant (frozen sine tests), bank widths $r = 128$–384 (two time integrators each), 50 time steps per query, six development cases with three repeats on one H200 GPU. Left: median solve time is unchanged from 256^2 to 4096^2 ($22.1/22.0/21.1$ ms at $r = 128$, $51.7/51.6/51.8$ ms at $r = 256$, $87.8/86.6/88.3$ ms at $r = 384$). Middle: Levenberg–Marquardt iterations per query are the same at every mesh. Right: only writing the six output fields grows with the mesh (0.13 to 5.7 ms at $r = 128$). At width 384 with backward Euler the worst-case error changes by 0.014 percentage points across these meshes. This is scaling evidence, not a speed claim: in the same study this variant was slower than the best full solver at matched accuracy at 4096^2 (0.05–$0.71\times$), with the gap closing as the mesh grows. The width-512 bank was timed at 1024^2 only.

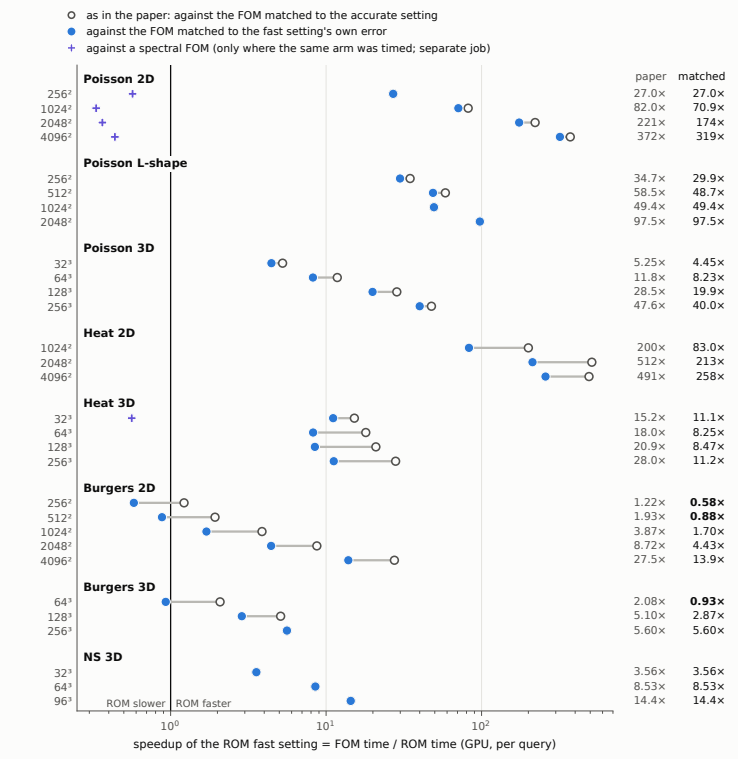


Figure G.4: Matched speedups. For the 30 cells of the submission's Table 1: speedup of the fast setting over the submission's named solver (filled) and over the rung of the same solver matched to its own error (open). Violet: transform or pseudo-spectral solvers where measured. Three cells fall below 1 (Burgers 2D 256^2 and 512^2, Burgers 3D 64^3). The Navier–Stokes cells are against CNAB2 on the same mesh; against a solver free to coarsen its mesh see §5.3.

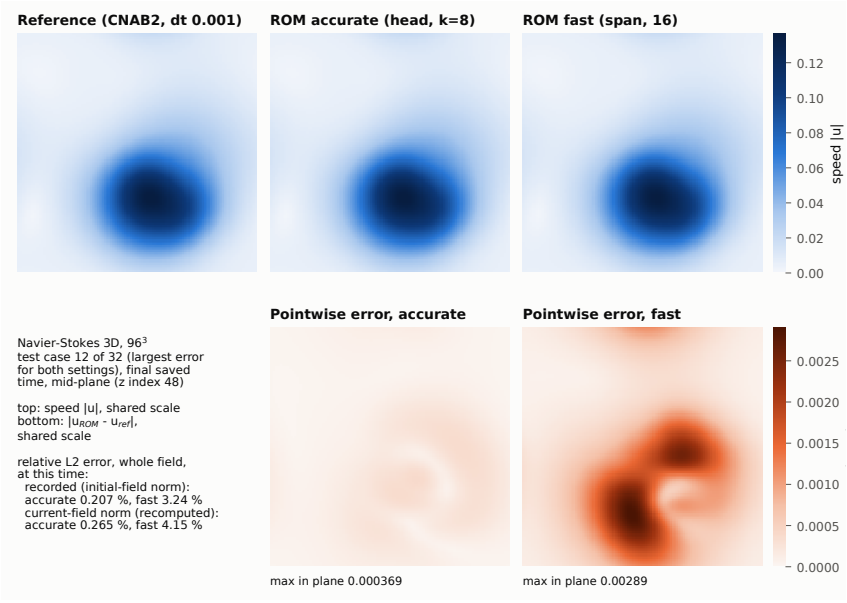


Figure G.5: Solution field and error map, original Navier–Stokes family at 96^3: speed $|u|$ on the mid-plane at $t = 0.2$ for the test case with the largest error. Top: reference, head ($k = 8$), span ($R' = 16$); bottom: $|u_{\mathrm{ROM}} - u_{\mathrm{ref}}|$, one shared scale.

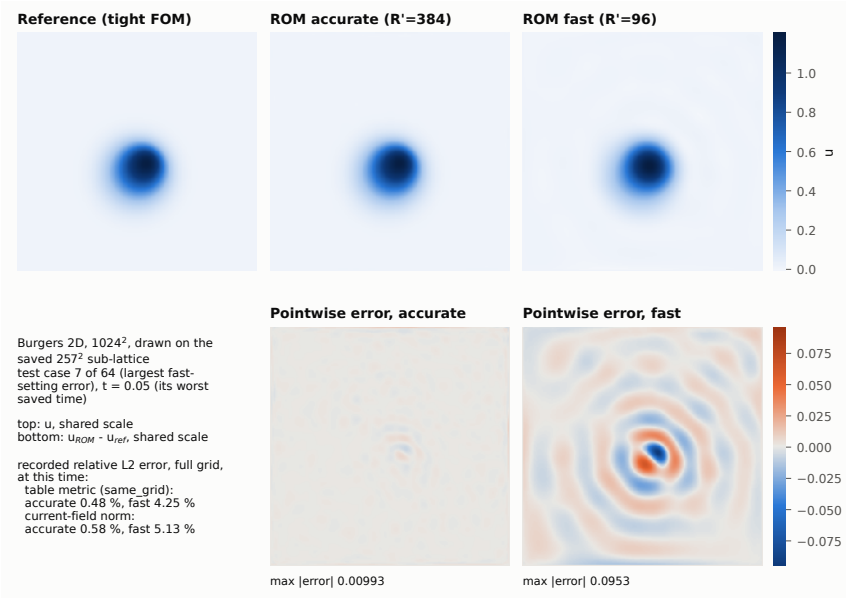


Figure G.6: Solution field and error map, Burgers 2D at 1024^2, $t = 0.05$, held-out case 7 (the fast setting's worst case), on the stored 257^2 restriction. Top: reference, accurate ($R' = 384$), fast ($R' = 96$); bottom: signed pointwise error.

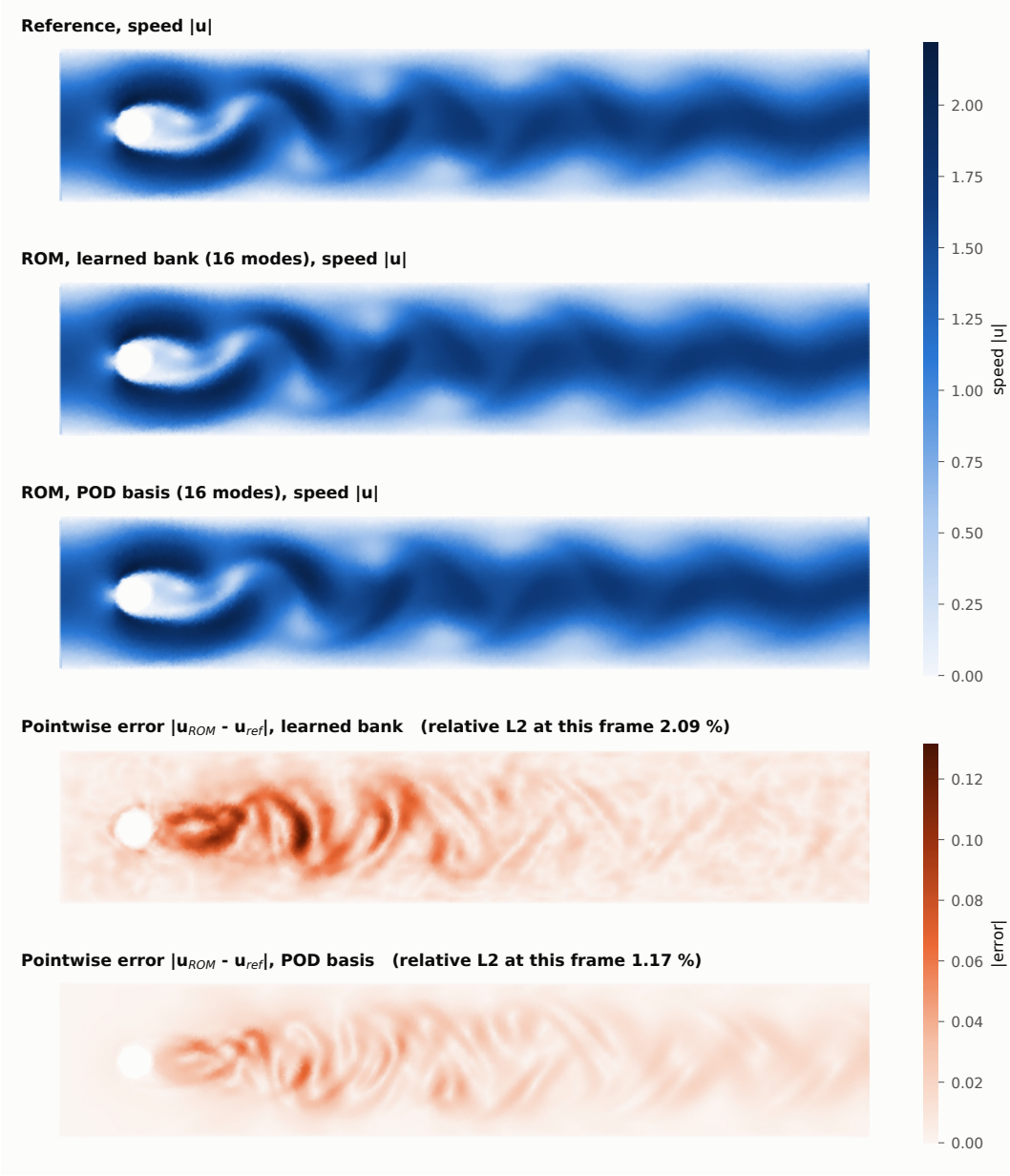


Figure G.7: Solution field and error map, cylinder flow, development Reynolds number 95, last stored frame: reference, a width-16 bank and a width-16 POD basis (top), velocity error magnitude (bottom). This rendering uses the first retrained bank of an earlier campaign, which is less accurate than POD here (2.1 against 1.2 % at this frame); the bank of Table 2 matches POD within 1 %.



Figure G.8: Learned basis functions: ordered bank functions 1–6, 32 and 128 of the frozen $R = 512$ Poisson 2D bank, each scaled by its own maximum, and the singular values that define the ordering. Low indices are smooth and large-scale; high indices carry fine scales.

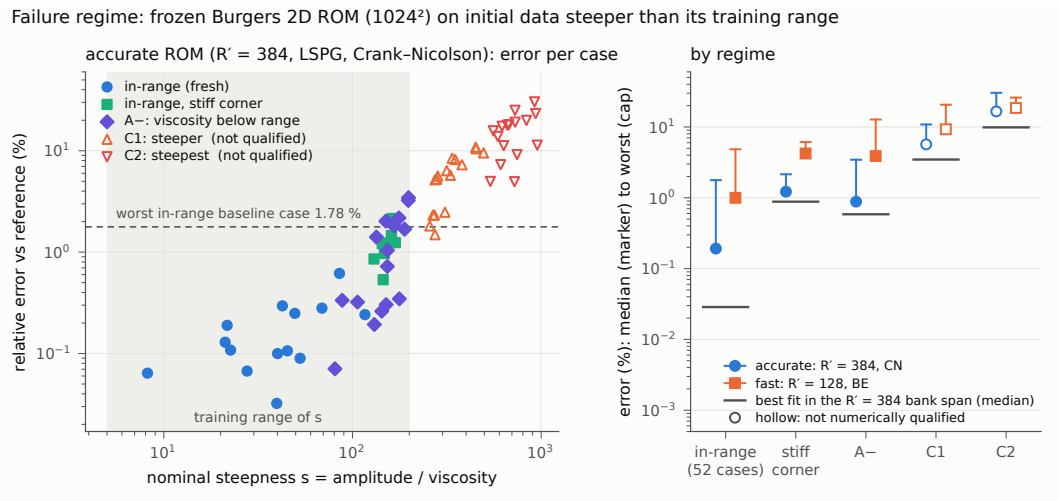


Figure G.9: Failure case: Burgers 2D at 1024^2 on initial data steeper than the training range (steepness s = amplitude/viscosity; training range shaded). Left: error per case of the accurate setting against s; hollow markers: regimes where the solve did not pass numerical qualification. Right: median to worst error per regime for the accurate and fast settings; grey: best fit in the bank's span. Medians rise to 5.7 % and 16.6 % on the two steepest exploratory development regimes, and the fit floor rises with them: the bank cannot represent these states. Metric: maximum over saved times relative to the initial norm.

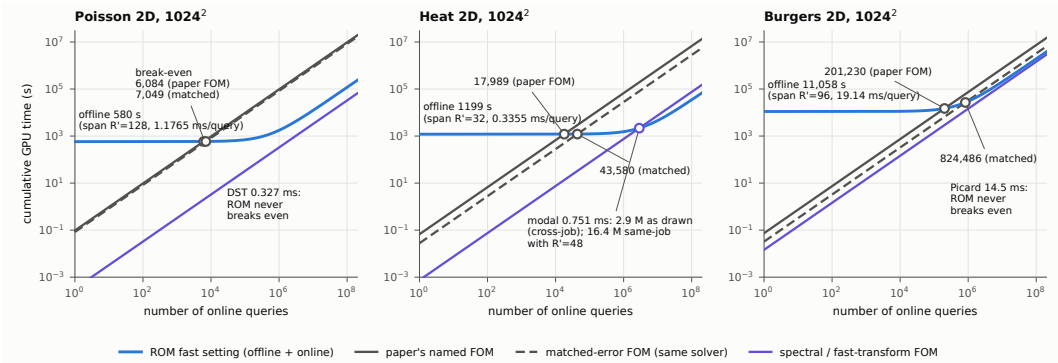


Figure G.10: Break-even curves: cumulative GPU time against number of queries at 1024^2 for Poisson 2D, heat 2D and Burgers 2D. Blue: fast setting including offline cost; grey solid: the submission's solver; grey dashed: matched solver; violet: transform or pseudo-spectral solver (never crossed for Poisson and Burgers at this mesh). Largest-mesh counts are in Table C.4.